



GRADO EN CIENCIA DE DATOS



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

DESCOMPOSICIÓN EN VALORES SINGULARES

PARA COMPRESIÓN DE IMÁGENES Y OTROS ALGORITMOS

COMPLEMENTARIOS

AUTOR: DAVID GARCERÁN HERRÁIZ

TUTORA: LEILA LEBTAHI CHEROUATI

JULIO 2020



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

DESCOMPOSICIÓN EN VALORES SINGULARES PARA COMPRESIÓN DE IMÁGENES Y OTROS ALGORITMOS COMPLEMENTARIOS

AUTOR: DAVID GARCERÁN HERRÁIZ

TUTORA: LEILA LEBTAHI CHEROUATI

JULIO 2020

TRIBUNAL

PRESIDENTE/A:

VOCAL 1:

VOCAL 2:

FECHA DE DEFENSA:

CALIFICACIÓN:

Declaración de autoría:

Yo, David Garcerán Herráiz, declaro la autoría del Trabajo Fin de Grado titulado “Descomposición en Valores Singulares para compresión de imágenes y otros algoritmos complementarios.” Y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 7 de junio de 2022

Fdo: David Garcerán Herráiz

Resumen:

La descomposición en valores singulares (SVD) es una de las factorizaciones matriciales más importantes, ampliamente utilizada para reducción de dimensionalidad, encontrar el rango de una matriz, calcular de forma rápida la inversa de una matriz, proporcionar fórmulas simples para ciertas expresiones que involucran normas matriciales, como el método de mínimos cuadrados.

En este trabajo se estudiará su aplicación en la compresión de imágenes digitales, también se presentarán algoritmos complementarios para mejorar su rendimiento y campos de aplicación donde vemos su utilidad.

Metodología

Conclusiones.

ÍNDICE:

Introducción	11
Objetivos	11
Conceptos Previos.....	12
Descomposición en Valores Singulares.....	17
Ejemplo de cálculo de la SVD de una matriz.....	21
Aplicación en imágenes.....	23
Rendimiento.....	31
Número de Valores Singulares k con respecto a la dimensión de la imagen	32
Calidad de la reconstrucción	34
Otros algoritmos complementarios	36
Híbrido SVD-WDR.....	36
Shuffled SVD.....	39
SVD en bloques	40
Aproximación de las matrices U, Σ y V	41
Optimización no lineal de k	44
Campos de Aplicación: ejemplos prácticos.....	47
Conclusiones y Proyección Futura	50
Anexos.....	51
Referencias.....	69

INTRODUCCIÓN

La descomposición en valores singulares (SVD) es una de las factorizaciones matriciales más importantes. Es una herramienta de gran utilidad, tanto en la práctica como en la teoría. Por ejemplo, en la práctica, la SVD se usa a menudo para encontrar el rango de una matriz, calcular de forma rápida la inversa de una matriz. También, la SVD puede proporcionar fórmulas simples para ciertas expresiones que involucran normas matriciales, como el método de mínimos cuadrados. Entre las muchas aplicaciones de la SVD, una de las más impresionantes es su uso en la compresión de imágenes digitales para que puedan ser transmitidas electrónicamente de manera eficiente (por satélite, fax, Internet o similares). Cada imagen en color o en escala de grises se representa con una matriz. El objetivo de este trabajo es estudiar cómo se utiliza la SVD para comprimir imágenes digitales.

OBJETIVOS

- Presentar la factorización SVD, junto a su razonamiento matemático y explicando los conceptos previos necesarios que se usarán en las explicaciones.
- Presentar la aplicación de la SVD en imágenes y cómo puede utilizarse para comprimirlas.
- Estudiar el rendimiento, tanto la tasa de compresión como la calidad de la reconstrucción.
- Proponer algoritmos complementarios que incluyan la SVD y mejoren su rendimiento.
- Exponer casos prácticos donde los algoritmos presentados son de utilidad.

CONCEPTOS PREVIOS

En esta parte se definen conceptos y resultados necesarios para este trabajo.

Espacio vectorial

Un espacio vectorial sobre un cuerpo $\mathbb{K}$ es una estructura algebraica creada a partir de un conjunto no vacío (V), una operación interna (llamada suma) y una operación externa (llamada producto por un escalar).

La operación interna tiene que cumplir las siguientes propiedades:

- Conmutativa: $u + v = v + u, \forall u, v \in V$.
- Asociativa: $u + (v + w) = (u + v) + w, \forall u, v, w \in V$.
- Elemento neutro: $\exists e \in V: u + e = u, \forall u \in V$.
- Elemento opuesto: $\forall u \in V, \exists -u \in V: u + (-u) = e$.

La operación externa tiene que cumplir las propiedades:

- Asociativa: $a \cdot (b \cdot u) = (a \cdot b) \cdot u, \forall a, b \in \mathbb{K}, \forall u \in V$.
- Elemento neutro: $\exists e \in \mathbb{K}: e \cdot u = u, \forall u \in V$.
- Distributiva respecto de la suma vectorial: $a \cdot (u + v) = au + av, \forall a \in \mathbb{K}, \forall u, v \in V$.
- Distributiva respecto de la suma escalar: $(a + b) \cdot u = a \cdot u + b \cdot u, \forall a, b \in \mathbb{K}, \forall u \in V$.

A los elementos de un espacio vectorial se les llama vectores y a los elementos del cuerpo se les conoce como escalares.

Notación:

Se denotará por $\mathbb{K}$ tanto a $\mathbb{R}$, el espacio vectorial de los reales, como a $\mathbb{C}$ el conjunto de los complejos, $\mathbb{N}$ el conjunto de los naturales y por $\mathbb{K}^{m \times n}$ al conjunto de las matrices de orden $m \times n$ con valores en $\mathbb{K}$.

Se denotará por $\mathbb{R}^+$ al conjunto: $\{x \in \mathbb{R}: x \geq 0\}$

Matriz identidad

Es la matriz cuadrada, denotada por I_n , que cumple la propiedad de ser el elemento neutro del producto de matrices. La columna i -ésima de esta matriz es el vector unitario e_i de una base vectorial inmersa en un espacio euclídeo de dimensión n . Por ejemplo:

$$I_3 = \text{diag}(1,1,1) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Matriz diagonal

Es una matriz cuadrada cuyos elementos fuera de la diagonal principal son todos cero:

La matriz $D = (d_{i,j})$ con n columnas y n filas es diagonal si:

$$d_{i,j} = 0 \text{ si } i \neq j \quad \forall i, j \in \{1, 2, \dots, n\}.$$

La multiplicación con un vector resulta ser el producto de cada elemento del vector por el elemento correspondiente a la diagonal:

$$D = \text{diag}(d_1, \dots, d_n), \quad v = [v_1 \dots v_n]^T.$$

$$Dv = \begin{bmatrix} d_1 & & \\ & \ddots & \\ & & d_n \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} = \begin{bmatrix} d_1 v_1 \\ \vdots \\ d_n v_n \end{bmatrix}.$$

Para la suma y multiplicación entre diagonales:

$$\begin{aligned} D + D &= \text{diag}(d_1 + d_1, \dots, d_n + d_n), \\ D^2 &= \text{diag}(d_1^2, \dots, d_n^2). \end{aligned}$$

Matriz traspuesta

Sea $A \in \mathbb{R}^{m \times n}$, la matriz traspuesta A^t está dada por:

$$(A^t)_{ij} = A_{ji}, \quad 1 \leq i \leq n, \quad 1 \leq j \leq m.$$

Propiedades

- $(A^t)^t = A$
- $(A + B)^t = A^t + B^t$
- $(cA)^t = cA^t$
- $(AB)^t = B^t A^t$

Matriz simétrica:

Sea $A \in \mathbb{R}^{n \times n}$, decimos que es simétrica cuando es cuadrada y además es idéntica a su traspuesta:

$$A = A^T.$$

Matriz ortogonal:

Sea $n \in \mathbb{N}$ y sea $A \in \mathbb{R}^{n \times n}$, se dice que A es ortogonal si:

$$A \cdot A^T = I_n.$$

Matriz no negativa:

Sea $A \in \mathbb{R}^{m \times n}$, decimos que es no negativa si:

$$(A)_{ij} \in \mathbb{R}^+, \quad 1 \leq i \leq n, \quad 1 \leq j \leq m.$$

Matriz semidefinida positiva

Es una matriz simétrica real cuyos menores principales son no negativos (Criterio de Sylvester).

Los menores principales se definen como el determinante de la matriz superior izquierda de dimensión $i \times i$, $i \in \{1, 2, \dots, n\}$.

- Si $A \in \mathbb{R}^{n \times n}$ entonces $A^t A$ es semidefinida positiva.

Rango de una matriz:

Se define el rango de una matriz $A \in \mathbb{R}^{m \times n}$ como el número máximo de columnas/filas que son linealmente independientes, anotado como $\text{rang}(A)$.

Por tanto:

$$0 \leq \text{rang}(A) \leq \min(m, n).$$

En general, se calcula mediante el método de Gauss Jordan, y será igual al número de filas no nulas de la matriz obtenida.

Subespacios fundamentales

Sea $A \in \mathbb{K}^{m \times n}$ ($\mathbb{K} = \mathbb{R}$ ó $\mathbb{C}$) una matriz de coeficientes $a_{ij} \in \mathbb{K}$. Se define el **espacio columna**, **espacio fila** y **espacio nulo** de A, respectivamente, como:

$$\text{Col}(A) = \text{gen}\{[a_{11} \dots a_{m1}]^t, \dots, [a_{1n} \dots a_{mn}]^t\},$$

$$\text{Fil}(A) = \text{gen}\{[a_{11} \dots a_{1n}]^t, \dots, [a_{m1} \dots a_{mn}]^t\},$$

$$\text{Nul}(A) = \{x \in \mathbb{K}^n: A \cdot x = 0_{\mathbb{K}^m}\},$$

donde 'gen' significa el espacio generado por vectores.

Ortogonalidad:

En un espacio vectorial euclídeo V , dos vectores $x, y \in V$ son ortogonales si el producto escalar $\langle x, y \rangle$ es nulo. Denotado como $x \perp y$:

$$x \perp y \Leftrightarrow \langle x, y \rangle = 0.$$

El producto escalar para dos vectores $x, y \in \mathbb{R}^n$ se define como:

$$\langle x, y \rangle = \sum_{i=1}^n x_i y_i$$

siendo x_i, y_i las i -ésimas componentes de x, y respectivamente.

Conjunto ortonormal

Un conjunto de vectores se dice ortonormal si es un conjunto ortogonal y la norma de cada uno de sus vectores es igual a 1. Sólo tiene sentido si los vectores pertenecen a un espacio vectorial en el que es definido un producto interno (producto escalar), como sucede en los espacios euclídeos.

El módulo o norma euclídea de un vector $x \in \mathbb{R}^n$ queda definido como:

$$||x||_2 = \sqrt{\sum_{i=1}^n x_i^2}.$$

Por tanto:

- $||x|| \geq 0; ||x|| = 0 \Leftrightarrow x = 0$
- $||x + y|| \leq ||x|| + ||y||$
- $||\alpha x|| = |\alpha| \cdot ||x|| \quad \forall \alpha \in (-\infty, +\infty)$

Por ejemplo, en el espacio euclídeo tridimensional $E^3 = (\mathbb{R}^3, \cdot)$, el conjunto

$S = \{(1,0,0), (0,1,0), (0,0,1)\}$ es un conjunto ortonormal.

Función

Dados dos conjuntos A y B , una función entre ellos es una asociación f que a cada elemento de A le asigna un único elemento de B . Se dice que A es el dominio de f , y que B es su codominio.

$$\begin{aligned} f: A &\rightarrow B \\ b &= f(a); \quad a \in A, b \in B. \end{aligned}$$

Función diferenciable

Una función de múltiples variables $f: \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}^m$ se dirá diferenciable en $x_0 \in \mathbb{R}^n$ si, siendo Ω un conjunto abierto en $\mathbb{R}^n$, existe una transformación lineal T , que cumpla:

$$f(x_0 + h) = f(x_0) + T(h) + \theta(h).$$

Donde $\theta(h)$ cumple que:

$$\lim_{h \rightarrow 0} \left(\frac{||\theta(h)||}{||h||} \right) = 0.$$

Por tanto:

$$\lim_{h \rightarrow 0} \frac{||f(x_0 + h) - f(x_0) - T(h)||}{||h||} = 0.$$

Transformación lineal

Se denomina transformación lineal, aplicación lineal, función lineal u operador lineal a toda función cuyo dominio y codominio sean espacios vectoriales, que preserve las operaciones de suma de vectores y multiplicación por un escalar. Dicho de otra forma, es un homomorfismo entre espacios vectoriales.

Sean V y W espacios vectoriales sobre el mismo cuerpo $\mathbb{K}$. Una aplicación T de V en W , es decir, $T: V \rightarrow W$, es una transformación lineal si para todo par de vectores $u, v \in V$ y para todo escalar $k \in \mathbb{K}$, se satisface que:

1. $T(u + v) = T(u) + T(v)$.
2. $T(ku) = kT(u)$.

Resumiendo las ecuaciones con dos escalares $\alpha, \beta \in \mathbb{K}$:

$$T(\alpha u + \beta v) = \alpha T(u) + \beta T(v).$$

Valor propio y Vector propio

Los vectores propios o auto vectores de una transformación lineal son los vectores no nulos que, cuando son transformados, dan lugar a un múltiplo escalar de sí mismos; es decir, no cambia su dirección. Este escalar λ recibe el nombre de valor propio, autovalor o valor característico.

Formalmente, sea $T: V \rightarrow V$ una transformación lineal en un cierto $\mathbb{K}$ -espacio vectorial V y v un vector no nulo en V . Si existe un escalar λ tal que:

$$T(v) = \lambda v, \quad v \neq 0, \quad \lambda \in \mathbb{K},$$

entonces v es un vector propio del operador T , y su valor propio asociado es λ .

DESCOMPOSICIÓN EN VALORES SINGULARES

DEFINICIONES PREVIAS

Dada una matriz $A \in \mathbb{R}^{m \times n}$:

La matriz $A^t A$ es simétrica: $(A^t A)^t = A^t (A^t)^t = A^t A$.

La matriz $A^t A$ es semidefinida positiva: $\langle x, A^t A x \rangle = x^t A^t A x = (Ax)^t Ax = \|Ax\|^2 \geq 0$.

Los autovalores de la matriz cuadrada, simétrica y semidefinida positiva $A^t A \in \mathbb{R}^{n \times n}$ son siempre reales y mayores o iguales a 0.

Definición:

Sean $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ los autovalores de la matriz $A^t A$ ordenados de mayor a menor. Entonces $\sigma_i = \sqrt{\lambda_i}$ es el i -ésimo **valor singular** de la matriz A.

Teorema:

Sean $\lambda_1 \geq \dots \geq \lambda_r > \lambda_{r+1} = \dots = \lambda_n = 0$ los primeros r autovalores no nulos, ordenados de manera decreciente y los $n - r$ autovalores nulos.

Sea $\{v_1, \dots, v_n\}$ una base ortonormal de $\mathbb{R}^n$ formada por autovectores de $A^t A$. Entonces:

1. $\{Av_1, \dots, Av_r\}$ es un conjunto ortogonal y $\|Av_i\| = \sqrt{\lambda_i} = \sigma_i$.
2. $\left\{\frac{Av_1}{\sigma_1}, \dots, \frac{Av_r}{\sigma_r}\right\}$ es una base ortonormal del subespacio fundamental $\text{Col}(A)$.
3. $\{v_{r+1}, \dots, v_n\}$ es una base ortonormal del subespacio fundamental $\text{Nul}(A)$.
4. $\text{rang}(A) = r$, es decir, el rango de la matriz A coincide con la cantidad de Valores Singulares no nulos.

Demostración:

1. $\langle Av_i, Av_j \rangle = v_i^t A^t A v_j = \lambda_j \cdot v_i^t v_j = \begin{cases} \lambda_j & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$
 $\|Av_i\| = \sqrt{\langle Av_i, Av_i \rangle} = \sqrt{\lambda_i} = \sigma_i$.
2. Como el conjunto $v_i, 1 \leq i \leq r$, es ortonormal, por lo tanto, linealmente independiente, se ve que $\frac{Av_i}{\sigma_i}$ es una combinación lineal de las columnas de A; por lo que el espacio generado por estos productos es el mismo que el generado por las columnas.
3. Si los vectores $v_i, r+1 \leq i \leq n$, están asociados a los autovalores nulos, sabiendo que $\text{Nul}(A) = \text{Nul}(A^t A)$, entonces $\{v_{r+1}, \dots, v_n\}$ es una base ortonormal de $\text{Nul}(A)$.
4. Dado que el conjunto $v_i, 1 \leq i \leq r$, es ortonormal, y mirando la dimensión del subespacio $\text{Col}(A)$, podemos ver que el rango de A es r.

Teorema:

La Descomposición en Valores Singulares (SVD) de una matriz $A \in \mathbb{R}^{m \times n}$ es una factorización del tipo:

$$A = U\Sigma V^T$$

con $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ ortogonales y $\Sigma \in \mathbb{R}^{m \times n}$ una matriz formada por los Valores Singulares (σ_i) de A en su diagonal principal ordenados de mayor a menor.

Demostración:

Sean $\lambda_1 \geq \dots \geq \lambda_r > \lambda_{r+1} = \dots = \lambda_n = 0$ los autovalores de $A^T A \in \mathbb{R}^{n \times n}$ ordenados de esta manera.

Sea $\{v_1, \dots, v_n\}$ una base ortonormal de $\mathbb{R}^n$ formada por los autovectores de $A^T A$, cada uno asociados en orden a un autovalor.

Recordemos que el conjunto $\{Av_1, \dots, Av_n\}$ es ortogonal, con $Av_{r+1} = \dots = Av_n = 0_{\mathbb{R}^m}$.

Si llamamos $\{u_1 = \frac{Av_1}{\sigma_1}, \dots, u_r = \frac{Av_r}{\sigma_r}\}$ vemos que es un conjunto ortonormal. Entonces, si $r < m$ podemos completar con $\{u_{r+1}, \dots, u_m\}$ hasta formar una base ortonormal de $\mathbb{R}^m$.

$$\begin{cases} Av_1 = \sigma_1 u_1 \\ \vdots \\ Av_r = \sigma_r u_r \\ Av_{r+1} = 0_{\mathbb{R}^m} \\ \vdots \\ Av_n = 0_{\mathbb{R}^m} \end{cases}$$

Reescribiéndolo en forma matricial con las matrices $V = [v_1 \dots v_n] \in \mathbb{R}^{n \times n}$ ortogonal y la matriz ortogonal U formada por las columnas ortonormales $\{u_1, \dots, u_r, u_{r+1}, \dots, u_m\}$, se tiene:

$$U\Sigma = [u_1 \dots u_m] \begin{bmatrix} \sigma_1 & 0 & \dots & 0 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \sigma_r & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & \dots & 0 \end{bmatrix} = [\sigma_1 u_1 \dots \sigma_r u_r \ 0 \dots 0].$$

Es decir, $U\Sigma = AV$.

Como V es ortogonal se puede escribir como $A = U\Sigma V^T$.

También se puede expresar A como combinación lineal de matrices de rango 1:

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T .$$

Observaciones:

- **$Rang(A) = Rang(A^T A) = Rang(AA^T)$.**

Sea $x \in Null(A)$, definido previamente el espacio nulo.

$$Ax = 0$$

$$A^T Ax = 0 .$$

Por tanto, $x \in Null(A^T A) \rightarrow Null(A) \subseteq Null(A^T A)$

$$x^T A^T Ax = 0$$

$$||Ax||_2^2 = 0$$

$$Ax = 0.$$

Por tanto $Null(A^T A) \subseteq Null(A) \rightarrow Null(A) = Null(A^T A)$.

Finalmente, tienen mismas dimensiones y mismo rango.

$$Rang(A) = Rang(A^T) = Rang(A^T A)$$

$$Rang(A^T) = Rang((A^T A)^T)$$

$$Rang(A) = Rang(AA^T).$$

- **$AA^T, A^T A$ tienen los mismos valores propios no nulos.**

Sea v un autovector asociado al autovalor λ no nulo de $A^T A$.

$$A^T A v = \lambda v$$

$$AA^T Av = A\lambda v$$

$$AA^T x = \lambda x, \quad \text{donde } x = Av .$$

Por tanto, λ también es autovalor de AA^T , y los valores singulares de A pueden calcularse usando cualquiera de estas dos matrices.

- **Σ es real no negativa**

Sea v un autovector asociado al autovalor λ no nulo de $A^T A$.

$$A^T Av = \lambda v$$

$$v^T A^T Av = \lambda v^T v$$

$$||Av||^2 = \lambda ||v||^2 \rightarrow \lambda \geq 0 .$$

Como las normas son siempre positivas, λ no puede ser negativa.

- Las filas de la matriz V^T son los vectores propios asociados a los valores propios de $A^T A$

$$\begin{aligned} A &= U\Sigma V^T \\ A^T &= (U\Sigma V^T)^T = V\Sigma^T U^T = V\Sigma U^T \\ A^T A &= V\Sigma U^T U\Sigma V^T = V\Sigma^2 V^T \\ A^T A V &= V\Sigma^2. \end{aligned}$$

Notar que $U^T U = I_m$, y $V^T V = I_n$ por ser matrices ortogonales y dado que Σ es una matriz diagonal, $\Sigma^T = \Sigma$.

- Las columnas de U son los vectores propios asociados a los valores propios de AA^T

$$\begin{aligned} A &= U\Sigma V^T \\ A^T &= V\Sigma U^T \\ AA^T &= U\Sigma^2 U^T \\ AA^T U &= U\Sigma^2. \end{aligned}$$

EJEMPLO DE CÁLCULO DE LA SVD DE UNA MATRIZ

Calculemos la SVD de la matriz A:

$$A = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

El primer paso sería calcular los autovalores λ_i y autovectores u_i de AA^T para calcular las columnas de U. Se calculan igualando el polinomio característico a 0, ya que:

$$\begin{aligned} AA^T u &= \lambda u \\ (AA^T - \lambda I)u &= 0 \\ u &\leftarrow \frac{u}{|u|} \end{aligned}$$

$$AA^T = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}.$$

Su polinomio característico sería $\begin{vmatrix} 1-\lambda & 1 \\ 1 & 1-\lambda \end{vmatrix} = (1-\lambda)^2 - 1 = \lambda^2 - 2\lambda = \lambda(\lambda - 2)$,

cuyas raíces son $\lambda_1 = 2$ y $\lambda_2 = 0$, asociados a los autovectores de norma unitaria

$u_1 = \frac{1}{\sqrt{2}} [1 \ 1]^T$ y $u_2 = \frac{1}{\sqrt{2}} [-1 \ 1]^T$. Nuestro único valor singular no nulo es $\sigma_1 = \sqrt{2}$.

Por tanto, tenemos que $U = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$.

Para calcular V, haremos lo mismo para $A^T A$:

$$A^T A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

Su polinomio característico sería $\begin{vmatrix} -\lambda & 0 & 0 \\ 0 & -\lambda & 0 \\ 0 & 0 & 2-\lambda \end{vmatrix} = \lambda^2(2-\lambda)$, cuyas raíces son:

$$\lambda_1 = 2, \lambda_2 = \lambda_3 = 0.$$

Sabiendo que $Av_i = \sigma_i u_i$, entonces $v_i = \frac{A^T u_i}{\sigma_i}$.

Para el único valor singular no nulo: $v_1 = \frac{1}{2} [0 \ 0 \ 2]^T = [0 \ 0 \ 1]^T$.

Completamos en una base ortonormal de $\mathbb{R}^3$: $\{[0 \ 0 \ 1]^T, [1 \ 0 \ 0]^T, [0 \ 1 \ 0]^T\}$. Por tanto:

$$V^T = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{y} \quad \Sigma = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

Finalmente:

$$A = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{2} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

SVD REDUCIDA

Es una variante en la que las matrices U_r, V_r, Σ_r sólo se construyen con los r Valores Singulares no nulos. En el ejemplo anterior:

$$U_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad V_1 = [0 \ 0 \ 1] \quad \Sigma_1 = \sqrt{2},$$

$$A = \frac{\sqrt{2}}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} [0 \ 0 \ 1] = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

APLICACIÓN EN IMÁGENES

Relación del álgebra lineal con las imágenes:

Las matrices tienen vital importancia hoy en día, por ejemplo, se usan para los gráficos por ordenador: la pantalla es una matriz de píxeles y multiplicando o sumando por las matrices adecuadas se pueden hacer rotaciones, traslaciones u otros movimientos con los gráficos.

Las matrices se usan para modelizar infinitud de problemas incluyendo las imágenes.

Una imagen es una matriz de píxeles, se puede formular como una función de 2 variables $f(x, y)$ de intensidad luminosa denominada nivel de gris, donde x e y son las coordenadas espaciales y el valor $f(x, y)$ es proporcional a la luminosidad de la imagen en el punto (x, y) .

$$f(x, y) = \begin{pmatrix} f(0,0) & f(0,1) & \cdots & f(0, n-1) \\ f(1,0) & f(1,1) & \cdots & f(1, n-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(m-1,0) & f(m-1,1) & \cdots & f(m-1, n-1) \end{pmatrix}$$

Para trabajar con imágenes matriciales, la función imagen $f(x, y)$ debe ser digitalizada.

Podemos entender las operaciones de rotación y escalado como transformaciones lineales, y por tanto se pueden representar con una matriz. Un ejemplo:

Teniendo la matriz $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$, cualquier punto (p, q) pasaría a ser el punto $(3p, 4p + 5q)$ en otro plano, dado por la operación: $\begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \cdot (p, q)^T$.

Gráficamente:

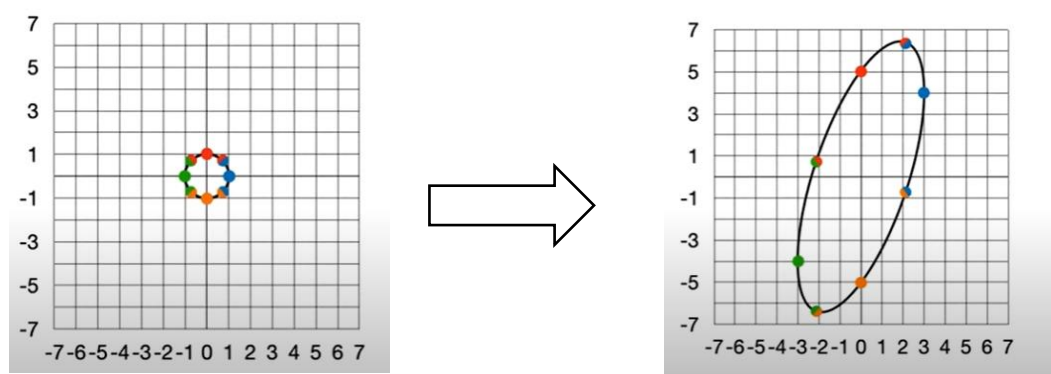


Ilustración 1 Extraído de: <https://youtu.be/DG7YTIGnCEo>

Si hacemos la descomposición SVD de la matriz, tenemos que las matrices U y V^t , son matrices de rotación del tipo:

$$\begin{bmatrix} -\sin(\theta) & \cos(\theta) \\ \cos(\theta) & \sin(\theta) \end{bmatrix}.$$

Siendo θ el ángulo de rotación. Y la matriz Σ , una matriz diagonal 2 x 2, definiéndose como matriz de escalado. La primera componente de la diagonal es el escalado en el eje horizontal y la segunda componente de la diagonal es el escalado en el eje vertical. En el caso anterior quedaría como:

$$\begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} -\sin(161.565^\circ) & \cos(161.565^\circ) \\ \cos(161.565^\circ) & \sin(161.565^\circ) \end{bmatrix} \begin{bmatrix} 6.708 & 0 \\ 0 & 2.236 \end{bmatrix} \begin{bmatrix} -\sin(135^\circ) & \cos(135^\circ) \\ \cos(135^\circ) & \sin(135^\circ) \end{bmatrix}.$$

Composición de las imágenes digitales:

Imagen digital:

Es una representación bidimensional de una imagen a partir de una matriz numérica. Dependiendo de si la resolución de la imagen es estática o dinámica, puede tratarse de una imagen matricial (mapa de bits) o de un gráfico vectorial.

Las imágenes se pueden obtener de varias formas:

- Por medio de dispositivos de entrada conversión analógica-digital como los escáneres y cámaras digitales
- Mediante programas informáticos editores de mapas de bits y dibujo vectorial.

Estructura:

La mayoría de los formatos de imágenes digitales están compuestos por una cabecera que contiene atributos como la dimensión, tipo de codificación, etc., seguida de los datos de la imagen en sí misma. Además, nos podemos encontrar una zona de metadatos. La forma de almacenar la cabecera y los datos depende de cada formato.

Resolución de la imagen:

Para imágenes digitales almacenadas como mapa de bits, es una tupla de dos enteros, donde el primero es la cantidad de columnas de píxeles (ancho) y el segundo la cantidad de filas de píxeles (alto).

También se le conoce como el número total de píxeles (ancho x alto) habitualmente expresado como el múltiplo correspondiente al millón / mega.

La resolución de impresión:

La resolución de impresión es la cantidad de píxeles por pulgada (PPP o DPI); 300 PPP se considera bueno.

Tamaño del archivo digital:

Obviando las diferentes maneras de compresión de imágenes que pueda tener cada tipo de formato, definiremos el tamaño como: píxeles en ancho x píxeles en alto x profundidad del píxel.

Profundidad de color:

Se conoce a la profundidad al número de bits que se concede a cada píxel para crear el color correspondiente. Así:

Si la imagen tiene profundidad de 1, significa que cada píxel se podrá codificar sólo con un bit, esto son 2 valores (0 y 1), lo que se traduce a una imagen en blanco y negro. Asignándose los píxeles con valor 0 para el negro y los píxeles con valor 1 para el blanco, o viceversa.

Si la imagen tiene profundidad de 1 byte (8 bits), significa que cada píxel podrá codificar $2^8 = 256$ niveles de gris, y tendremos una imagen en escala de grises.

Imagen en color:

Para representar una imagen a color, asociaremos a cada píxel una tupla de n enteros, cada posición de la tupla representa un canal. Por tanto, el canal es similar a la imagen en escala de grises sobre un color primario. Según los distintos modelos de color:

- **RGB:**
Se compone de 3 canales: rojo, verde y azul. Habitualmente utilizado en monitores de ordenador y escáneres de imagen.
El tamaño convencional (desde 2005) es de 24 bits, 8 bits por canal, cada uno codificando 256 niveles.
- **CMYK:**
Se compone de 4 canales: cian, magenta, amarillo y negro. Habitualmente utilizado en impresión.
El tamaño convencional (desde 2005) es de 32 bits, 8 bits por canal.
- **HSV:**
Se compone de 3 canales: tono, saturación y luminosidad. Habitualmente utilizado en compresión de vídeo con pérdida, donde la pérdida de información de color es menos perceptible al ojo humano.
- **$YC_B C_R$:**
Se compone de 3 canales: Y representa la componente de luminosidad, C_B, C_R son los componentes de crominancia diferencia de azul y diferencia de rojo, respectivamente.
No es un espacio de color absoluto, sino una forma de codificar información RGB.
Es usada en sistemas de vídeo y fotografía digital.

Para imágenes sin fondo, se utiliza el canal alfa, que almacena la información de transparencia. Habitualmente usado en composición de imágenes digitales.

Optimización del tamaño del canal:

Como el cerebro no percibe necesariamente diferencias en cada canal para el mismo grado que en otros canales, es posible que cambiar el número de bits para cada canal resulte en un almacenamiento óptimo; en particular, para imágenes RGB, comprimir más el canal azul y después el rojo puede ser mejor que dar el mismo espacio a cada canal.

Profundidad	Tipo de imagen	Número de colores	Ejemplo
1 bit	Blanco y Negro	2	
8 bits	Escala de grises	256	
24 bits	Color	256	
32 bits	Transparencias	256	
48 bits	Color	65536 (2^{16})	

Tabla 1

Digitalización:

La digitalización consiste en la transcripción de señales analógicas, en nuestro caso imágenes reales, en señal digital. Podemos considerar que la información sobre el color que podemos percibir con el ojo humano tiene carácter continuo; al digitalizar una imagen, limitamos la cantidad de colores que podemos percibir con la profundidad de cada píxel.

Como realmente no podemos distinguir infinitos colores, podemos reducir el número de colores representados sin apreciar cambios. Por ejemplo, teniendo los siguientes píxeles [1,0,0] [0,1,0] y [0,0,1] en RGB con 24 bits, no podríamos diferenciarlos del negro puro ([0,0,0]).

Partiendo de la imagen y usando su matriz correspondiente, se pretende simplificar la matriz para obtener que la mayoría de las entradas sean ceros.

Uno de los métodos para lograrlo es la SVD, y a esto se le llama la compresión de imágenes reduciendo el tamaño de archivos digitales.

Compresión de imágenes:

Distinguimos dos tipos de compresiones de imágenes según conserven la totalidad de información de la imagen, así tenemos los métodos sin pérdida (*lossless compression*) que se caracterizan por permitir recuperar exactamente la calidad original de la imagen. Y, por el contrario, métodos con pérdida (*lossy compression*) los cuales se caracterizan por desprenderse de cierta información de la imagen original de cara a una mayor compresión.

Las técnicas más comunes para métodos con pérdida se basan en reducir el espacio de color a los colores más comunes en la imagen, reduciendo así el número de bits para codificarlos, en promediar o eliminar parte de la información cromática de la imagen o en una transformación que suelen ser la transformada discreta de coseno (*DCT*), la transformada de Fourier (*FT*) o la transformada de ondícula (*DWT*).

La compresión que estudiaremos a continuación se basa en eliminar los valores singulares menos importantes una vez aplicada la SVD a una imagen, es decir, será un método con pérdidas.

Compresión de imagen utilizando la Descomposición en Valores Singulares:

Para ilustrar el uso de la Descomposición en Valores Singulares para compresión de imágenes, vamos a tomar como referencia la siguiente imagen:



- Dimensiones: 297 x 264 píxeles
- Profundidad en bits: 32
- Tipo de elemento: Archivo PNG

Ilustración 2 Extraída de:

<https://www.uv.es/recursos/fatwirepub/ccurl/46/266/GCD.png>

Se trata de una imagen a color con transparencias, es decir, tiene 4 canales: RGB y canal alfa, de 8 bits cada uno. Para aplicar el ejercicio vamos a considerar sólo los 3 canales RGB, es decir, que ahora la profundidad pasará a ser 24 bits. Obviando la compresión que tienen los archivos PNG, nuestra imagen modelo debería tener un tamaño de $297 \times 264 \times 24 = 1881792$ bits, lo que vienen a ser 229.71 KB.



Ilustración 3

Podemos entender cada canal como una matriz, típicamente en RGB se representa con 8 bits de profundidad, es decir que cada píxel puede representar $2^8 = 256$ colores. Entonces, lo que en una escala de grises el 255 sería blanco y el 0 negro, en color el blanco sería la tupla $[255, 255, 255]$ y el negro $[0, 0, 0]$.

Leeremos la imagen en el espacio RGB, pero la transformaremos al espacio YCbCr para asegurar una buena calidad de reconstrucción, una vez transformada debemos aplicar la SVD a una nueva matriz $A = [Y, C_B, C_R]^T$, siendo $Y, C_B, C_R \in \mathbb{N}^{264 \times 297}$ las diferentes bandas de la transformada:

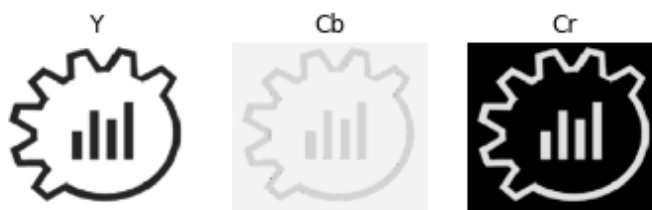


Ilustración 4

Si calculamos la factorización $A = U\Sigma V^T$ para cada canal, nos quedamos con que las dimensiones de las matrices son: $U \in \mathbb{R}^{792 \times 792}$, $V^T \in \mathbb{R}^{297 \times 297}$ y $\Sigma \in \mathbb{R}^{792 \times 297}$

Observamos la raíz octava de la magnitud de los Valores Singulares de la diagonal Σ :

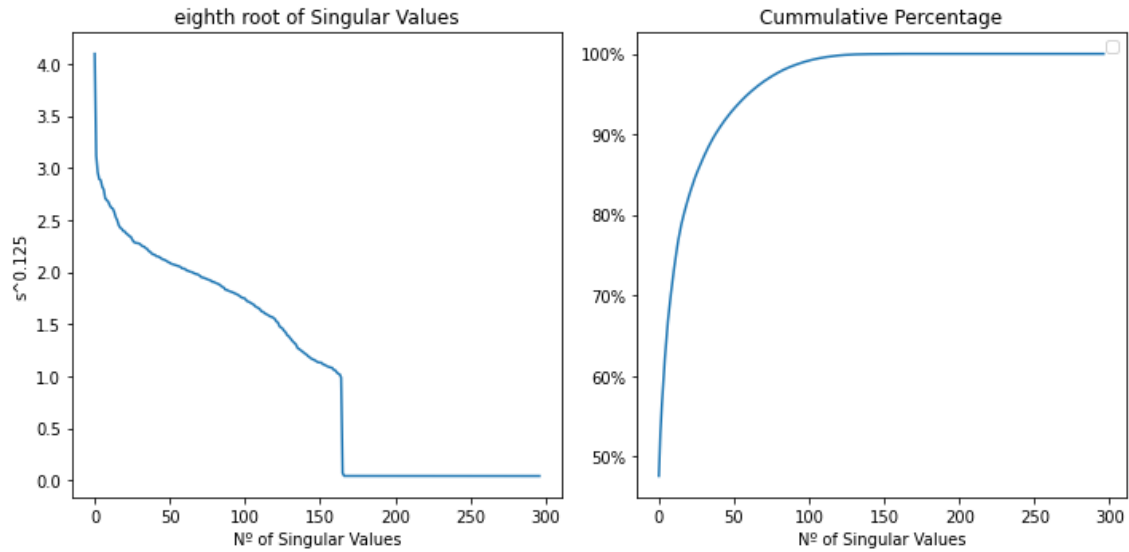


Ilustración 5

Como observamos, a partir de los 154 primeros valores singulares la desviación estándar es mínima (2.49×10^{-12}). Vamos a truncar la matriz Σ para asignar estos últimos valores a 0 y reconstruimos la imagen:



- Dimensiones: 297 x 264 píxeles
- Profundidad en bits: 24

Ilustración 6

Vamos a ilustrar cómo es reconstruir la imagen con distintas cantidades de Valores Singulares:



Ilustración 7

RENDIMIENTO

El objetivo de comprimir imágenes usando la SVD es que consigamos que, guardando las 3 matrices de la factorización, consigamos que ocupe menos memoria que guardando la imagen como tal.

Teniendo en cuenta sólo el número de entradas (una entrada equivaldría a un píxel en imágenes en blanco y negro o escala de gris), y tomando una imagen cualquiera, de tamaño $m \times n$: $A \in \mathbb{N}^{m \times n}$, en escala de grises, es decir, con un solo canal, los píxeles ocupados serían $m \times n$.

Si aplicamos la SVD y truncamos para k valores singulares, $\hat{A} = U\Sigma V^T$ podemos utilizar la forma reducida: $U \in \mathbb{R}^{m \times k}$, $V^T \in \mathbb{R}^{k \times n}$ y $\Sigma \in \mathbb{R}^{k \times k}$. Y así, el número de entradas sería:

$$mk + k + kn = k(m + n + 1).$$

Podemos guardar la matriz Σ como un vector de dimensión k ya que es una matriz diagonal.

Suponiendo que existe compresión:

$$m \times n > k(m + n + 1)$$

$$k < \frac{m \times n}{m + n + 1},$$

para ilustrar el crecimiento de k (en la altura) en función de m, n .

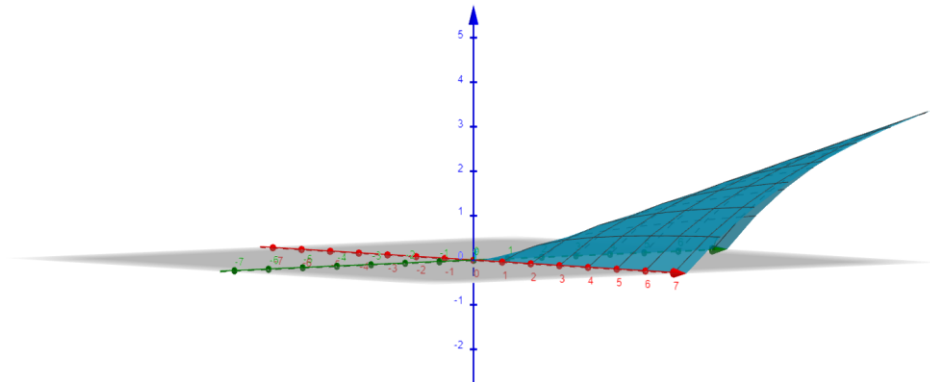


Ilustración 8 Creado con Geogebra: <https://www.geogebra.org/graphing?lang=es>

Nótese, que $m, n, k \in \mathbb{N}$, y que este gráfico es meramente ilustrativo.

Número de Valores Singulares k con respecto a la dimensión de la imagen

En este punto vamos a explorar la proporción que supone el número de valores singulares k que elegimos para truncar la SVD, con respecto a las dimensiones de la imagen $m \times n$.

Primero vamos a suponer que la imagen tiene dimensión $n \times n$, es decir la matriz será cuadrada.

Recordamos la frontera para la que k hace que sea eficiente la compresión, $k < \frac{n \times n}{n+n+1}$, y la comparamos con n :

$$\lim_{n \rightarrow \infty} \frac{\left(\frac{n^2}{2n+1} \right)}{n} = \lim_{n \rightarrow \infty} \frac{n}{2n+1} = \lim_{n \rightarrow \infty} \frac{1}{2} = 0.5.$$

Si resolvemos mediante la regla de l'Hôpital, obtenemos que, para valores grandes de n , la frontera para k tiende a $\frac{n}{2}$.

Ahora vamos a estudiarlo para cuando la imagen no es cuadrada, es decir, tiene dimensión $m \times n$:

$$\text{Sea } f(m, n) = \frac{m \times n}{m+n+1}.$$

$$\begin{aligned} \lim_{n, m \rightarrow \infty} \frac{f(m, n)}{n} &= \frac{m}{m+n}, \\ \lim_{n, m \rightarrow \infty} \frac{f(m, n)}{m} &= \frac{n}{m+n}. \end{aligned}$$

En resumen, la proporción de $\frac{k}{n}$ tiende a m' en el vector normalizado $(m', n') = \frac{(m, n)}{|(m, n)|}$ y viceversa.

Retomamos los cálculos del rendimiento y la tasa de compresión, con la imagen A con un canal, a nivel de gris.

Si tenemos en cuenta la memoria que ocupan y no sólo el número de entradas, los resultados cambian. Ya que la imagen original se compone de números naturales, concretamente de 0 a 255, y, por tanto, cada píxel de esta imagen ocupa 8 bits.

Guardar la imagen como tal ocuparía $m \times n \times 8$ bits.

Si guardáramos las 3 matrices de la factorización SVD hay que tener en cuenta que los números reales ocupan mucha más memoria que los números naturales, en concreto:

- Si hablamos del formato en coma flotante de simple precisión, utilizaremos 32 bits para cada número, y tendrá de 6 a 7 dígitos significativos.
- Si hablamos del formato en coma flotante de doble precisión, utilizaremos 64 bits para cada número, y tendrá de 15 a 16 dígitos significativos.

En nuestro caso utilizaremos el formato en coma flotante de simple precisión, 32 bits, y, por tanto, nuestras 3 matrices ocuparán: $k(m + n + 1) \times 32$ bits.

Nuestra nueva frontera para k es la siguiente:

$$\begin{aligned} m \times n \times 8 &> k(m + n + 1) \times 32 \\ k &< 0.25 \cdot \frac{m \times n}{m + n + 1}, \end{aligned}$$

reduciendo bastante el número máximo de valores singulares que hacen efectiva la compresión. Y que en algunos casos supone una pérdida de información visualmente significativa.

Por ejemplo, en el caso de una matriz cuadrada, para n grande, $k < \frac{n}{8}$.

En el caso de que sea una imagen a color, guardar la imagen como tal costaría: $m \times n \times 24$.

Mientras que guardar las tres matrices en forma corta U_k, V_k, Σ_k de la matriz

$$A = \begin{cases} [Y, C_B, C_R]^T & \text{si } m < n \\ [Y, C_B, C_R] & \text{si } m > n \end{cases}$$

costaría: $32k(3 \min(m, n) + \max(m, n) + 1)$, siendo k el número de valores singulares al truncar la SVD. Si asumimos que hay compresión:

$$\begin{aligned} 24 \times m \times n &> 32k(3 \min(m, n) + \max(m, n) + 1) \\ k &< 0.75 \frac{m \times n}{3 \min(m, n) + \max(m, n) + 1}. \end{aligned}$$

Si nos ponemos en el caso de una matriz imagen cuadrada:

$$\lim_{n \rightarrow \infty} 0.75 \cdot \frac{n}{4n + 1} = \frac{3}{16}.$$

Por tanto, en el caso de una matriz cuadrada, para n grande, $k < \frac{3}{16}n$. Que ya es más grande que en el caso anterior, ya que no hemos escalado la SVD para los tres canales.

Calidad de la reconstrucción

Para cuantificar la calidad de la reconstrucción en el ámbito de la compresión de imágenes, usaremos la medida PSNR (*Peak Signal-to-Noise Ratio*) que define la relación entre la máxima energía posible de una señal y el ruido que afecta a su representación fidedigna. El PSNR se expresa generalmente en escala logarítmica, utilizando como unidad el decibelio.

Así, el PSNR se define como:

$$PSNR = 10 \cdot \log_{10} \left(\frac{(2^B - 1)^2}{MSE} \right) = 20 \cdot \log_{10} \left(\frac{2^B - 1}{RMSE} \right),$$

siendo B la profundidad del píxel, y $RMSE$ la raíz del error cuadrático medio, que para dos imágenes monocromáticas $A, \hat{A} \in \mathbb{N}^{m \times n}$ se define como:

$$RMSE = \sqrt{\frac{1}{m \cdot n} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (A_{i,j} - \hat{A}_{i,j})^2}.$$

En caso de que las imágenes sean a color, el RMSE se calcula como la media aritmética de los RMSE de los tres canales.

Si aplicamos esta medida a las pruebas que hicimos anteriormente sobre nuestra imagen modelo:

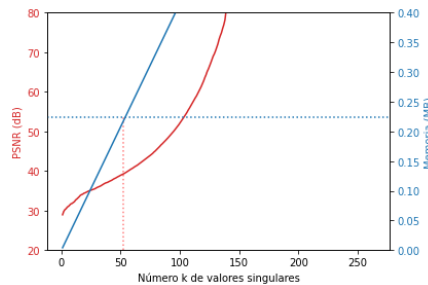


Ilustración 9

En la ilustración 9 se muestra con una línea punteada azul lo que ocuparía guardar la imagen en sí misma, mientras que la línea sólida es la memoria de la k-aproximación. En el otro eje tenemos el PSNR, y con una línea roja punteada, una guía para ilustrar qué PSNR obtendríamos en el mejor de los casos. Vemos que no aparece el PSNR de los últimos valores porque el error es casi 0.

Como vemos, a partir de 53 valores singulares ya ocuparía más que la imagen original, y el PSNR en este punto con 53 valores es de 39.55 dB; sin embargo, si observamos en toda la escala k, este método consigue un valor máximo con la mitad de los valores singulares. La única limitación que tenemos es la manera de almacenar. La imagen reconstruida en este punto es:

Nº de Valores Singulares= 53



Ilustración 10

Tomando como nueva imagen modelo la misma, pero en escala de grises, observamos el proceso de reconstrucción en función del número de valores singulares y la gráfica del PSNR:

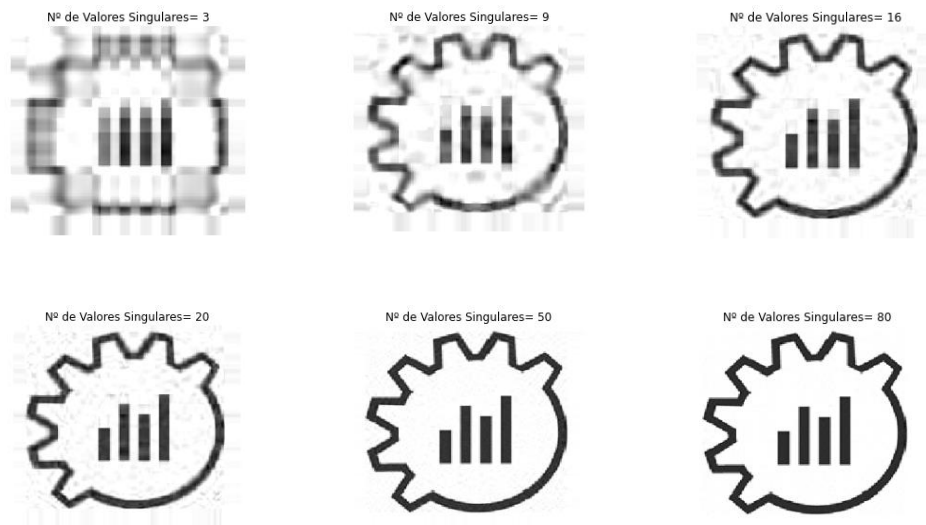


Ilustración 11

Como observamos, la imagen se aprecia en los primeros 50 valores, lo único que cambia a continuación es el color del fondo.

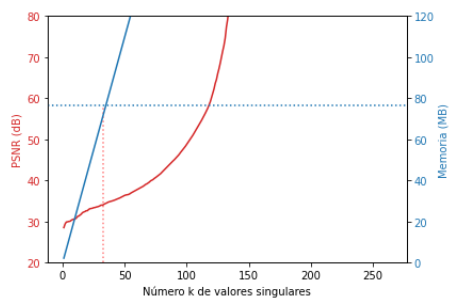


Ilustración 12

En la ilustración 12 se muestra con una línea punteada azul lo que ocuparía guardar la imagen en sí misma, mientras que la línea sólida es la memoria de la k-aproximación.

En el otro eje tenemos el PNSE, y con una línea roja punteada, una guía para ilustrar qué PNSE obtendríamos en el mejor de los casos.

Como vemos, a partir de 34 valores singulares ya ocuparía más que la imagen original, y el PSNR en este punto con 34 valores es de 34.45 dB; en comparación con la imagen a color, tenemos muy buenas medidas de PNSR. La imagen reconstruida en este punto es:



Ilustración 13

OTROS ALGORITMOS COMPLEMENTARIOS

Híbrido SVD-WDR

El algoritmo WDR (del inglés, *wavelet difference reduction*) es una técnica basada en ondículas, en la que primero, se aplica la Transformada ondícula discreta (DWT del inglés, *discrete wavelet transform*) y después se aplica un codificador de plano de bits.

DWT:

La transformada de ondícula discreta es un tipo especial de transformada matemática que representa una señal en términos de versiones trasladadas y dilatadas de una onda finita (denominada óndula madre). Dichas ondículas son muestreadas en un espacio discreto. Todas las transformaciones de ondículas pueden ser consideradas formas de representación en tiempo-frecuencia.

La ventaja frente a las transformadas de Fourier es la resolución temporal, ya que esta transformada captura la frecuencia e información sobre la posición. Es muy utilizada en análisis, eliminación de ruido y compresión de señales. En la práctica, la señal se pasa por dos filtros, uno pasa baja y otro pasa alta, tantas veces como el orden de la transformada, extrayendo así diferentes características de la imagen, como pueden ser, los bordes verticales, horizontales y diagonales, texturas ...

Este método trata de descomponer la imagen original en ondículas, representadas por sus coeficientes, entonces podemos codificar estos coeficientes de la siguiente manera:

Los coeficientes de la DWT, tomando como ejemplo un orden de 3, vienen ordenados de la siguiente manera

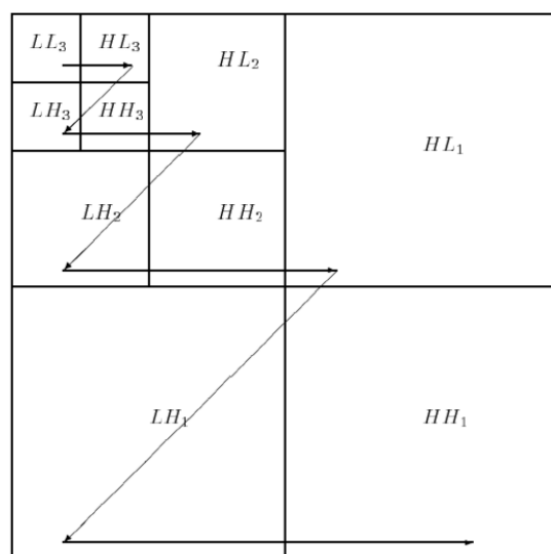


Ilustración 14

Así pues, en una descomposición de orden N , el escaneo empieza en LL_N , sigue por HL_N, LH_N, HH_N , en este punto pasa a un orden inferior: $LL_{N-1}, HL_{N-1}, LH_{N-1}, HH_{N-1} \dots$ en orden por cada subregión HL_i se hace por columnas, y en cada subregión LH_i se hace por filas.

Un coeficiente de la transformada x se define como significativo con respecto a un umbral T si $|x| \geq T$, en caso contrario se considera no significativo.

El algoritmo trata de iterar sobre el umbral T , buscando los índices significativos y guardando sus índices en una lista, que llamaremos ξ . Iniciando con un umbral tal que $|x_j| < 2T \forall j$ y $|x_{j_0}| \geq T$ para algún j_0 .

Teniendo la lista ξ , con los índices de los coeficientes significativos en esta iteración, que han sido eliminados de la lista original y movidos a otra. La codificaremos con la serie en diferencias ξ' , a excepción del primer número que será el mismo que en la original, así, empezaremos con el primer número, y para sacar la lista original habrá que hacer una suma acumulativa. Dado que esta lista es una sucesión siempre creciente, guardar la serie en diferencias es rentable.

Para pasar esta lista ξ' a un *bit stream*, seguiremos el siguiente formato: el primer valor será '+' o '-' dependiendo del signo del coeficiente al que correspondía, el segundo valor será '0' si el valor del coeficiente correspondiente $|x_i| < 1.5T$, y por el contrario '1'. Los siguientes valores serán el índice escrito en binario y eliminado el primer bit, conocido como MSB (*most significant bit*), ya que siempre será uno. Un ejemplo:

Como los índices comienzan en 1, el primer índice en binario se escribiría como $(1)_2$, si suponemos que el primer valor era significativo, positivo y menor que $1.5T$, lo tendremos codificado como: '+0', ya que hemos eliminado el MSB. Si dicho coeficiente estuviera en el índice 16, $(10000)_2$ se codificaría como: '+00000'.

El resultado de esta iteración será una cadena de índices codificados, usando por separadores los signos '+' y '-'.

Para la siguiente iteración, el umbral se reduce a la mitad, $T \leftarrow \frac{T}{2}$, y se opera con los coeficientes restantes.

Esto se repite unas 12 veces, quedando los datos codificados (usando el carácter '|' para separar entre iteraciones) y cuantificados, ya que, en la operación de decodificación, nos fijaremos en el segundo valor de cada índice codificado, si es '1' se le asignará el valor $1.5T_k$, y si es '0' se le asignará el valor T_k , siendo T_k el umbral en la iteración correspondiente, además el signo del coeficiente reconstruido viene dado por nuestros signos separadores, y la posición por el índice codificado.

Notar que todos los valores en cada iteración k están comprendidos entre $T_k \leq |x_i| \leq 2T_k$, y que al quedar cuantificados a T o $1.5T$, se consigue una buena aproximación.

Un ejemplo aplicado a nuestra imagen modelo en blanco y negro:

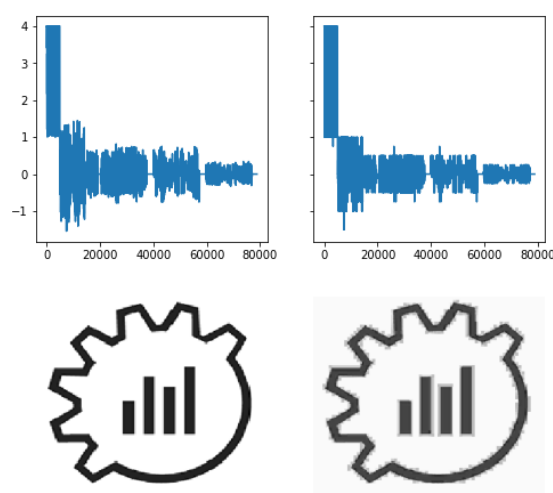


Ilustración 15

En la derecha vemos la imagen original, y arriba sus coeficientes de la transformada. A la izquierda, los coeficientes reconstruidos y debajo la imagen reconstruida. En la gráfica se observa el efecto de la cuantización, y que a magnitudes mayores que 1, pierde sensibilidad.

Pues el interés de este algoritmo es la tasa de compresión, y por eso lo combinaremos con el método de la SVD: buscaremos el número de valores singulares que conserven la mayoría de información de la imagen, sin importar la memoria que ocuparía guardar las matrices, reconstruimos la imagen y la pasamos al algoritmo WDR, el resultado será un '*bitstream*', que habrá que reconstruir para obtener la imagen truncada.

Dicho bitstream debemos codificarlo para reducir el número de bits por carácter y mediante el algoritmo *Huffman* (Rufai, Anbarjafari, & Demirel, 2013) obtenemos los distintos códigos para los caracteres:

CARACTER	CÓDIGO
0	0
1	11
+	101
-	1001
	1000

Tabla 2

Estos códigos se asignan mediante un árbol de decisión binario de tantos nodos finales como símbolos únicos. Se tienen en cuenta la frecuencia de aparición, así los caracteres que más se repiten tienen menor número de bits. Este método nos garantiza que podemos leer la cadena entera sin necesidad de separadores, ya que cada código no puede ser prefijo de ningún otro, así pues, para decodificar una cadena, basta con recorrer el árbol binario hasta llegar a un nodo.

Shuffled SVD

Este método consiste dividir la imagen original en bloques, el número de bloques irá determinado por las dimensiones de la imagen, en el mejor de los casos, la imagen tendrá que ser cuadrada y su altura será un cuadrado perfecto, es decir, tomando la imagen $A \in \mathbb{N}^{n \times n}$, $n = k^2$ $k \in \mathbb{N}$. En el caso de que no sea cuadrada, buscaremos que sus dimensiones sean cuadrados perfectos, y si no se da el caso, buscar el divisor más cercano a $\sqrt{n}$ y $\sqrt{m}$ respectivamente.

Una vez dividida la imagen en bloques, se permutarán siguiendo un mapeo S , $\hat{A} = S(A)$, y a esta imagen se le aplicará la conocida SVD, dando como resultado $\hat{A}_k$. El último paso es reordenar los bloques de la imagen, $A_k = S^{-1}(\hat{A}_k)$.

Con este método, conseguimos que las k-aproximaciones de pocos valores singulares eleven su calidad, ya que al menos, seguirán el orden de la permutación. Tiene un gran inconveniente, y es que al permutar la imagen podemos desaprovechar la ventaja de tener un bajo rango, y a su vez un peor rendimiento en la SVD. Por eso, para muchos valores singulares se puede apreciar ruido blanco en toda la imagen, teniendo así un peor rendimiento para muchos valores.

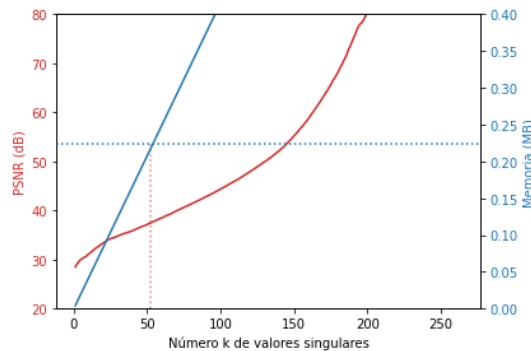


Ilustración 16

Si la comparamos con el rendimiento de la SVD explicada anteriormente, la curva tiene menor pendiente. Para $k=53$ el valor es 37.07 dB.

Breve comparación:

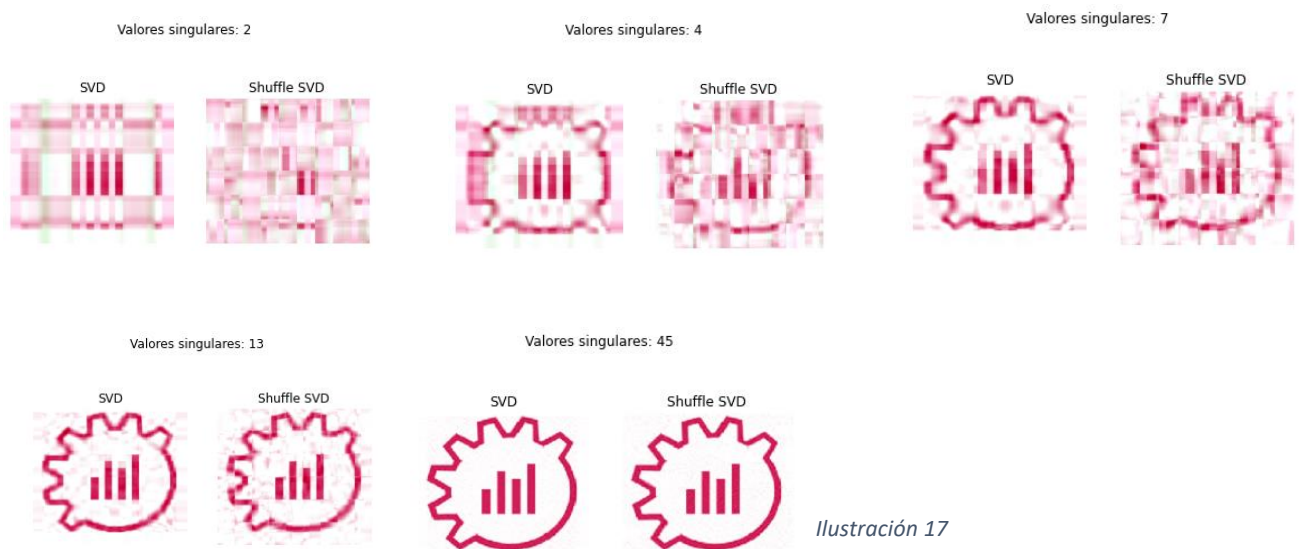


Ilustración 17

SVD en bloques

La estrategia es muy similar a la de 'Shuffled SVD' descrito en el apartado anterior. La imagen se divide en bloques. La diferencia es que no será necesario permutarlos ya que se aplicará una SVD a cada bloque individualmente. Lo interesante de esta variante es que, al seleccionar zonas de la imagen en bloques pequeños, reducimos la información contenida, siendo así más probable que dicho bloque tenga un rango muy bajo y la compresión SVD sea muy eficaz.

La memoria ocupada sería:

$$32\alpha\beta k \left(3 \min\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + \max\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + 1 \right), \text{ para RGB.}$$

$$32\alpha\beta k \left(\frac{m}{\alpha} + \frac{n}{\beta} + 1 \right), \text{ para escala de grises.}$$

Siendo α y β divisores de m y n respectivamente, entonces, la imagen se partirá en $\alpha\beta$ bloques de dimensión $\left(\frac{m}{\alpha}, \frac{n}{\beta}\right)$, siendo m y n las dimensiones de la imagen y k el número de valores singulares. En la imagen modelo, se eligieron $\alpha = 12, \beta = 11$, que son los divisores más cercanos a las raíces de las dimensiones.

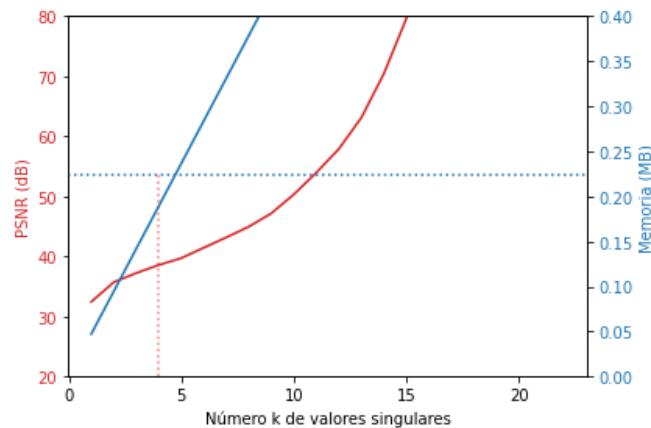


Ilustración 18

Como vemos, a partir de 4 valores singulares por bloque ya ocuparía más que la imagen original, y el PSNR en este punto con 4 valores es de 39.72 dB; sin embargo, si observamos en toda la escala k, este método consigue un valor máximo con la mitad de los valores singulares. La única limitación que tenemos es la manera de almacenar. La imagen reconstruida en este punto es:



Ilustración 19

Aproximación de las matrices U, Σ y V

En este punto veremos un método para reducir el tamaño que ocuparía guardar las matrices de cualquier factorización SVD. Podemos observar que los valores de las matrices U, V pertenecen al intervalo abierto $(-1,1)$, mientras que los valores de Σ pertenecen a los reales positivos ($\mathbb{R}^+$). Es decir:

$$(U)_{i,j} \in \{x \in \mathbb{R}: -1 < x < 1\}$$

$$(V)_{j,l} \in \{x \in \mathbb{R}: -1 < x < 1\}, \quad 1 \leq i \leq m, \quad 1 \leq j \leq k, \quad 1 \leq l \leq n.$$

La aproximación consiste en redondear los componentes de las matrices U y V a 4 decimales para así, multiplicándolos por 10^4 sólo en el proceso de almacenar, tratarlos como enteros de 16 bits; mientras que nos quedaremos con la parte entera de los valores singulares, es decir utilizaremos la función suelo: $\lfloor x \rfloor = \max\{k \in \mathbb{Z}: k \leq x\}$. Aunque se seguirán guardando en 32 bits porque las magnitudes de los primeros valores singulares no se pueden codificar con 16 bits, y es preferible escoger la siguiente potencia de 2.

Con esto conseguimos pasar de guardar las 3 matrices en 32 bits, a 2 de ellas en 16 y la más breve en 32. La memoria ocupada, y el valor frontera de k quedarían de la siguiente forma:

- Imagen en escala de grises:

$$k(16m + 16n + 32) = 16k(m + n + 2) \text{ bits.}$$

$$16k(m + n + 2) < 8mn$$

$$k < 0.5 \frac{m \times n}{m + n + 2}.$$

Asumiendo matriz cuadrada y n grande:

$$\lim_{n \rightarrow \infty} 0.5 \frac{n}{2n + 2} = 0.25.$$

Por tanto, en estas asunciones $k < \frac{1}{4}n$.

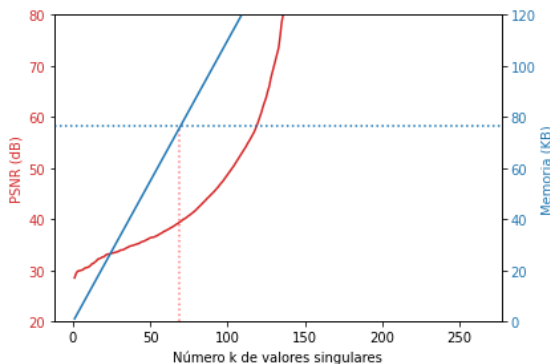


Ilustración 20

Nº de Valores Singulares= 69



En este caso, el número máximo k posible es 69 y se consigue 39.62 dB.

- Imagen en color:

$$k(48 \min(m, n) + 16 \max(m, n) + 32) = 16k(3 \min(m, n) + \max(m, n) + 2) \text{ bits.}$$

$$16k(3 \min(m, n) + \max(m, n) + 2) < 24mn$$

$$k < 1.5 \cdot \frac{m \times n}{3 \min(m, n) + \max(m, n) + 2}.$$

Asumiendo matriz cuadrada y n grande:

$$\lim_{n \rightarrow \infty} 1.5 \frac{n}{4n + 2} = 0.375.$$

Por tanto, en estas asunciones $k < \frac{3}{8}n$.

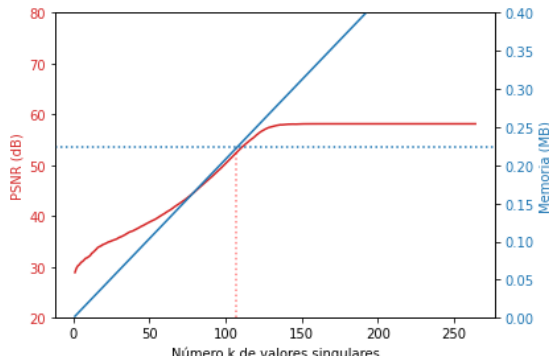


Ilustración 21

Nº de Valores Singulares = 107



En este caso, el número máximo k posible es 107 y se consigue 52.78 dB.

- SVD en bloques, RGB:

$$16\alpha\beta k \left(3 \min\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + \max\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + 2 \right) \text{ bits.}$$

$$16\alpha\beta k \left(3 \min\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + \max\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + 2 \right) < 24mn$$

$$k < \frac{1.5}{\alpha\beta} \cdot \frac{m \times n}{3 \min\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + \max\left(\frac{m}{\alpha}, \frac{n}{\beta}\right) + 2}.$$

Asumiendo matriz cuadrada y n grande:

$$\lim_{n \rightarrow \infty} \frac{1.5}{\alpha} \frac{n}{4n + 2\alpha} = \frac{3}{8\alpha}.$$

Por tanto, en estas asunciones $k < \frac{3n}{8\alpha}$.

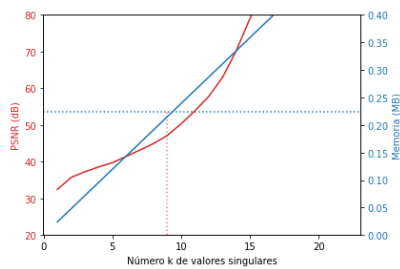


Ilustración 22

Nº de Valores Singulares = 9



En este caso, el número máximo k posible por bloque es 9 y se consigue 50.33 dB.

- SVD en bloques, escala de grises:

$$16\alpha\beta k \left(\frac{m}{\alpha} + \frac{n}{\beta} + 2 \right) \text{ bits.}$$

$$16\alpha\beta k \left(\frac{m}{\alpha} + \frac{n}{\beta} + 2 \right) < 8mn$$

$$k < \frac{1}{2\alpha\beta} \cdot \frac{m \times n}{\frac{m}{\alpha} + \frac{n}{\beta} + 2}.$$

Asumiendo matriz cuadrada y n grande:

$$\lim_{n \rightarrow \infty} \frac{0.5}{\alpha} \frac{n}{2n + 2\alpha} = \frac{1}{4\alpha}.$$

Por tanto, en estas asunciones $k < \frac{n}{4\alpha}$.

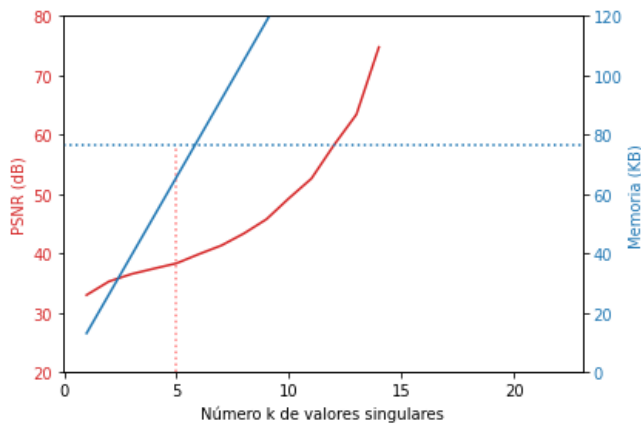


Ilustración 23



En este caso, el número máximo k posible por bloque es 5 y se consigue 39.9 dB.

En resumen, aproximar los valores de las matrices convenientemente para poder almacenarlos como enteros, y así reducir, aproximadamente, a la mitad el espacio ocupado, nos da posibilidad de compresión sin sacrificar calidad de la imagen. Y esto es lo que veremos en el siguiente punto, que trata de optimizar el valor K para maximizar la calidad y la compresión conseguida.

Optimización no lineal de k

Usaremos el optimizador no lineal de *Python* en '*scipy.optimize.minimize*', concretamente el método 'Powell'. Este es un algoritmo propuesto por Michael J.D. Powell para buscar un mínimo local de una función, la función no tiene que ser diferenciable y, por tanto, no se usan derivadas. La función debe ser de valores reales para un número fijo de entradas reales. (Powell, 1964)

El primer paso es construir una función de coste en función de k y que tenga en cuenta la tasa de compresión y la calidad de la k -aproximación. Esta será la función que minimizar:

$$\min_k \left(\sqrt{f(k)^2 - g(k)^2} \right)$$

$$\text{s. a:}$$

$$0 < k \leq \min(m, n).$$

$$f(k) = \begin{cases} 5 \cdot 10^4 \text{ mem}(A_k) & \text{si } \text{mem}(A_k) > \text{mem}(A). \\ \frac{\text{mem}(A_k)}{\text{mem}(A)} & \text{si } \text{mem}(A_k) \leq \text{mem}(A). \end{cases}$$

$$g(k) = \begin{cases} 5 \cdot 10^4 - \text{PSNR}(A_k) & \text{si } \text{PSNR}(A_k) < a. \\ \frac{b - \text{PSNR}(A_k)}{a - b} & \text{si } a \leq \text{PSNR}(A_k) \leq b. \\ 0 & \text{si } \text{PSNR}(A_k) > b. \end{cases}$$

Notación:

- $A \in \mathbb{N}^{m \times n}$, es la imagen original.
- $A_k \in \mathbb{N}^{m \times n}$, es la k -aproximación calculada por cualquier método de SVD expuesto.
- mem es la función que calcula la memoria ocupada, cambia según el método.
- PSNR es la medida de calidad ya expuesta.
- $a, b \in \mathbb{R}^+$ son el intervalo de decibelios que consideramos aceptable para la reconstrucción. En este caso serán $a = 36, b = 50$ dB.

En el caso de optimizar el algoritmo de la SVD en bloques, k será un vector en el que se puede asumir que todos los bloques usarán la misma k , u optimizar k_i para cada bloque. La ventaja de optimizar un k_i para cada bloque es poder reducir el número de entradas en aquellos bloques donde haya menor cantidad de información, como puede ser, una zona vacía.

Para la SVD normal:

Nº de Valores Singulares= 46



- Tasa de compresión: 14.20%
- PSNR: 38.26 dB

Ilustración 24

Para la aproximación de la SVD normal:

Nº de Valores Singulares= 66



- Tasa de compresión: 38.78%
- PSNR: 41.74 dB

Ilustración 25

Para la SVD por bloques, asumiendo misma k:

Nº de Valores Singulares= 4



- Tasa de compresión: 15.60%
- PSNR: 38.58 dB

Ilustración 26

Para la aproximación de la SVD por bloques, asumiendo la misma k :

Nº de Valores Singulares= 7



- Tasa de compresión: 25.36%
- PSNR: 43.18 dB

Ilustración 27

Para la SVD por bloques, con diferente k :

Nº de Valores Singulares variable



- Tasa de compresión: 17.04%
- PSNR: 38.40 dB

Ilustración 28

Para la aproximación de la SVD por bloques, con diferente k :

Nº de Valores Singulares variable



- Tasa de compresión: 34.01%
- PSNR: 40.78 dB

Ilustración 29

Los resultados de la optimización para los casos de la SVD por bloques, sin asumir misma k , no son los esperados. Aunque, en global, las medidas son buenas, se aprecian visualmente las erratas. Esto puede ser debido al método de resolución, que ajusta los valores del vector k heurísticamente y luego evalúa las medidas de memoria y PSNR, pero en ningún momento tiene información del contenido de la imagen, o la magnitud de los valores singulares...

CAMPOS DE APLICACIÓN: EJEMPLOS PRÁCTICOS

Este algoritmo puede ser de utilidad en diversos campos de aplicación, a continuación, se mostrarán las imágenes en las que se probará nuestro algoritmo junto a un breve comentario sobre la aplicación del algoritmo en dicho campo. Y finalmente, una tabla comparativa de los rendimientos.



Ilustración 30 Extraída de: <https://www.colgate.com/es-uy/oral-health/x-rays/what-is-a-panoramic-dental-x-ray>

En el campo sanitario, es de gran importancia que las imágenes adjuntas a los informes del paciente ocupen el menor espacio posible sin perder apenas calidad, ya que una pérdida de información severa imposibilitaría estudios como la detección del cáncer basado en la imagen. (Mirar en Datos óhmicos el formato ese, y explicar un poco).



Ilustración 31 Extraída de: https://es.wikipedia.org/wiki/Lunch_atop_a_Skyscraper

En el campo de las Redes Sociales, es importante maximizar la calidad de la reconstrucción, ya que es un criterio importante por el que valorarán los usuarios. Debido a la complejidad de la imagen en el sentido de cantidad de información, tomaremos como imagen ejemplo la famosa foto tomada en la construcción del edificio RCA, Nueva York, 1932.

DICCIONARIO ETIMOLÓGICO que en rigor merezca tal título, no lo posee hasta ahora lengua alguna, ni lo poseerá en mucho tiempo. En efecto, para llamarse con toda propiedad *etimológico* un Diccionario, además de contener la lista alfabética completa de las voces primitivas y simples, debería consignar respecto de cada una de ellas las particularidades siguientes:

1.ª Su etimología inmediata, ó, mejor dicho, su origen inmediato, su última procedencia, esto es, la indicación de la lengua de que se hubiese tomado ó proviniese inmediatamente, poniendo á continuación la voz de correspondencia ó la voz equivalente en dicha lengua.

2.ª En qué época se había tomado.

3.ª Su significación recta ó primitiva cuando fue admitida, justificándola con la cita de algún texto impreso, y aun manuscrito, siempre que fuese de autoridad competente.

4.ª La primera forma que en la pronunciación, y por escrito, tuvo aquella voz al tomarse de la lengua de origen inmediato, y las alteraciones ortográficas ó prosódicas que

Ilustración 32 Extraída de: <https://www.ecured.cu/Texto>

En el campo de reconocimiento de imágenes, más concretamente reconocimiento de texto, podemos encontrar diversos servicios o herramientas online que hacen uso de esto, como, por ejemplo, transcribir escritura manuscrita, interpretar fórmulas matemáticas u obtener el texto de la imagen. Las herramientas que se basen en un servicio en la nube requerirán que el usuario suba la imagen, por tanto, conlleva una compresión de la imagen.



Ilustración 33 Extraída de: <https://diariodegastronomia.com/imagenes-por-satelite-para-detectar-fraudes-en-el-riego/>

Las imágenes por satélite pueden ser un claro ejemplo de la necesidad de comprimir la imagen, ya que conlleva una transmisión de esta. Este tipo de imágenes es ampliamente utilizado en cartografía, sistemas de información geográfica (GIS), estudios del clima, los océanos, los vientos, la vegetación o corrientes globales; en el campo militar (tareas de reconocimiento, espionaje y seguimiento).

Imagen	Tipo de SVD	Tasa de compresión	PSNR
	SVD normal	13.26 %	37.89 dB
	SVD por bloques	25 %	34.18 dB
	Aprox. SVD normal	38.32 %	41.41 dB
	Aprox. SVD bloques	10.67 %	39.27 dB
	SVD normal	2 %	30.59 dB
	SVD por bloques	18.53 %	30.23 dB
	Aprox. SVD normal	15.83 %	32.70 dB
	Aprox. SVD bloques	18.15 %	31.26 dB
DICCIONARIO ETIMOLOGICO que en rigor merece tal titulo, no lo posee hasta ahora lengua alguna, ni lo poseerá en mucho tiempo. En efecto, para llamarse con toda propiedad etimológico un Diccionario, además de contener la lista alfabética completa de las voces primitivas y simples, debería consignar respecto de cada una de ellas las particularidades siguientes: 1.° Su etimología inmediata, ó, mejor dicho, su origen inmediato, su última procedencia, esto es, la indicación de la lengua de que se hubiese tomado ó proviniese inmediatamente, poniendo á continuación la voz de correspondencia ó la voz equivalente en dicha lengua. 2.° En que época se halla tomado. 3.° Su significación recta ó primitiva cuando fue admitida, justificándola con la cita de algun texto impreso, y aun manuscrito, siempre que fuese de autoridad competente. 4.° La primera forma que en la pronunciación, y por escrito, tuvo aquella voz al tomarse de la lengua de origen inmediato, y las alteraciones ortográficas ó prosódicas que	SVD normal	0.4 %	31.24 dB
	SVD por bloques	3.7 %	30.79 dB
	Aprox. SVD normal	0.3 %	33.04 dB
	Aprox. SVD bloques	19.5 %	31.26 dB
	SVD normal	34.56 %	28.44 dB
	SVD por bloques	20.45 %	26.31 dB
	Aprox. SVD normal	26.75 %	29.35 dB
	Aprox. SVD bloques	25.89%	28.56 dB

Tabla 3

CONCLUSIONES Y PROYECCIÓN FUTURA

Contar la idea básica de compresión con SVD

Exponer el límite de memoria

Decir que aproximar redondeando es muy buena solución.

Decir que en RGB funciona mejor que en gris

Proyección:

- Optimización basada en la magnitud de S
- Autoencoders

ANEXOS

Como anexos se adjuntan todo el código de las clases y funciones utilizadas, escrito en el lenguaje de programación *Python*, en el programa *Spyder*.

modulo_PSNR.py:

```
1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Mon Jun  6 13:26:01 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.
9.  import numpy as np
10. import matplotlib.pyplot as plt
11. from sklearn.metrics import mean_squared_error as MSE
12.
13.
14. class PSNR:
15.     '''
16.     Clase destinada a evaluar el PSNR de cualquier método SVD para
17.     distinto número de Valores Singulares.
18.     '''
19.
20.     def __init__(self, img, fun):
21.
22.         self.RGB = len(img.shape) > 2
23.         self.img=img
24.         self.fun=fun(img)
25.         if hasattr(self.fun, 'B'):
26.             self.max_k=min(self.fun.B.step_m,
27.                             self.fun.B.step_n)
28.         else:
29.             self.max_k=min(img.shape[:2])
30.         self.mem_lim=img.shape[0]*img.shape[1]/1024
31.         if self.RGB:
32.             self.mem_lim*=3/1024
33.
34.         return
35.     def RMSE(self,b):
36.         a=self.img
37.         if self.RGB:
38.             r=[0,0,0]
39.             for i in range(3):
40.                 r[i]=MSE(a[:, :, i], b[:, :, i])**0.5
41.             r=np.mean(r)
42.
43.         else:
44.             r=(MSE(a,b))**0.5
45.
46.         return r
47.
48.     def PSNR(self,b):
49.         a=self.img
50.         if self.RGB:
51.             r=[0,0,0]
52.             for i in range(3):
53.                 r[i]=20*np.log10(255/(MSE(a[:, :, i], b[:, :, i]))**0.5)
54.             r=np.mean(r)
55.
56.         else:
57.             r=20*np.log10(255/(MSE(a,b))**0.5)
58.
59.         return r
60.
61.     def sim(self):
```

```

62.         K=np.linspace(1, self.max_k,self.max_k).astype(int)
63.         e=np.zeros(len(K))
64.         for n in range(len(K)):
65.             truncated=self.fun.trunc(int(K[n]))
66.             e[n-1]=self.PSNR(truncated)
67.
68.         self.mem=np.array([self.fun.mem(int(k)) for k in K])
69.
70.
71.         corte=np.where(self.mem<=self.mem_lim)[0][-1]
72.
73.         if self.RGB:
74.             ylim=max(0.4,self.mem_lim)
75.             unit='MB'
76.         else:
77.             ylim=max(120,self.mem_lim)
78.             unit='KB'
79.
80.         fig,ax1=plt.subplots()
81.         color='tab:red'
82.         ax1.set_xlabel('Número k de valores singulares')
83.         ax1.set_ylabel('PSNR (dB)',color=color)
84.         ax1.plot(K,e,color=color)
85.         ax1.axvline(x=corte + 1,linestyle='dotted',color='red',
86.                     alpha=0.5,ymax=self.mem_lim/ylim)
87.         ax1.set_ylim(20,80)
88.         ax1.tick_params(axis='y',labelcolor=color)
89.
90.         ax2=ax1.twinx()
91.
92.         color='tab:blue'
93.         ax2.set_ylabel('Memoria (%s)'%unit,color=color)
94.         ax2.plot(K,self.mem,color=color)
95.         ax2.axhline(y=self.mem_lim,linestyle='dotted')
96.         ax2.tick_params(axis='y',labelcolor=color)
97.         ax2.set_ylim(0,ylim)
98.         fig.tight_layout()
99.         plt.show()
100.        print(e[corte])
101.
102.        _=self.fun.plot(int(corte)+1)
103.
104.        return
105.

```

modulo_SVD.py:

```

1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Mon Jun  6 14:15:41 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  from skimage import io,color,exposure
9.  import numpy as np
10. import matplotlib.pyplot as plt
11. from matplotlib.ticker import PercentFormatter
12. from modulo_PSNR import PSNR
13. import warnings
14. warnings.filterwarnings("ignore")
15.
16. def rescale(img,in_range,out_range=(0,255)):
17.     '''
18.     Parameters
19.     -----

```

```

20.     img : np.array
21.         imagen a escalar
22.     in_range : tupla
23.         rango de valores en los que esta img
24.     out_range : tupla, optional
25.         rango de valores de salida. The default is (0,255).
26.
27.     Returns
28.     -----
29.     Escala los valores de la imagen a de un rango a otro.
30.
31.     '''
32.
33.     return exposure.rescale_intensity(img,in_range=in_range
34.                                     ,out_range=out_range).round().astype('uint8')
35.
36.
37. class SVD_raw:
38.     """
39.     Clase destinada a ejecutar la SVD normal sobre cualquier imagen
40.     """
41.
42.     def __init__(self,img):
43.         self.RGB= len(img.shape)>2
44.         self.img=img
45.         if self.RGB:
46.             img=color.rgb2ycbcr(img)
47.             rows,cols,_=img.shape
48.             Y=img[:, :,0]
49.             cb=img[:, :,1]
50.             cr=img[:, :,2]
51.
52.             if rows<=cols:
53.                 A=np.array([Y,cb,cr]).reshape(rows*3,cols)
54.             else:
55.                 A=np.array([Y,cb,cr]).reshape(rows,cols*3)
56.
57.             self.u,self.s,self.v=np.linalg.svd(A,full_matrices=False)
58.
59.         else:
60.             self.u,self.s,self.v=np.linalg.svd(self.img,full_matrices=False)
61.
62.         self.S=self.s.copy()
63.
64.         return
65.     def trunc(self,k):
66.         """
67.         Parameters
68.         -----
69.         k : int
70.             Número de valores singulares.
71.
72.         Returns
73.         -----
74.         truncated : np.array
75.             Imagen reconstruida con esos valores singulares.
76.
77.         """
78.         u=self.u[:, :k]
79.         s=self.s[:k]
80.         v=self.v[:, :k]
81.         A= (u*s)@v
82.
83.         if self.RGB:
84.
85.             rows,cols,_=self.img.shape
86.             truncated=np.zeros((rows,cols,3))
87.             for i in range(3):
88.                 if rows<=cols:
89.                     truncated[:, :,i]=A[rows*(i):rows*(i+1),:]

```

```

90.         else:
91.             truncated[:, :, i] = A[:, cols*(i):cols*(i+1)]
92.             truncated = color.ycbcr2rgb(truncated)
93.             truncated = exposure.rescale_intensity(truncated, in_range=(0, 1)
94.                                                    , out_range='uint8').round().astype('uint8')
95.         else:
96.             truncated = rescale(A, (0, 255))
97.
98.         return truncated
99.
100.     def plot(self, k):
101.         """
102.         Muestra la imagen reconstruida con k valores singulares
103.
104.         Parameters
105.         -----
106.         k: int
107.             Número de valores singulares.
108.
109.         Returns
110.         -----
111.         None.
112.
113.         """
114.         fig, ax = plt.subplots(1, 2, figsize=(10, 5))
115.         ax[0].plot(self.S**0.125)
116.         ax[1].plot(np.cumsum(self.S)/sum(self.S))
117.         ax[0].set_ylabel('s^0.125')
118.         ax[0].set_xlabel('Nº of Singular Values')
119.         ax[0].set_title('eighth root of Singular Values')
120.         ax[1].yaxis.set_major_formatter(PercentFormatter(xmax=1))
121.         ax[1].set_title('Cumulative Percentage')
122.         ax[1].set_xlabel('Nº of Singular Values')
123.
124.         fig.tight_layout()
125.
126.         plt.legend()
127.         plt.show()
128.         plt.title('Nº de Valores Singulares= %i'%(k))
129.         if self.RGB:
130.             plt.imshow(self.trunc(k))
131.         else:
132.             plt.imshow(self.trunc(k), cmap='gray')
133.         plt.axis('off')
134.
135.     def mem(self, k):
136.         """
137.         Calcula la memoria que ocuparía la reconstrucción
138.
139.         Parameters
140.         -----
141.         k: int
142.             Número de Valores Singulares.
143.
144.         Returns
145.         -----
146.         m: float
147.             Memoria usada, MB si es a color, KB si no.
148.
149.         """
150.
151.         if self.RGB:
152.             rows, cols, _ = self.img.shape
153.
154.             m = k*(max(rows, cols)+3*min(rows, cols)+1)*32/8/1024**2 #MB
155.         else:
156.             rows, cols = self.img.shape
157.             m = k*(rows+cols+1)*32/8/1024 #Kb
158.
159.         return m

```

```

160.
161. def main():
162.     #RGB
163.     img=img=io.imread('../GCD.PNG')[::,:3]
164.     A=PSNR(img,SVD_raw)
165.     A.sim()
166.
167.     #gray
168.     gray=color.rgb2gray(img)
169.     gray=rescale(gray,(0,1))
170.     A=PSNR(gray,SVD_raw)
171.     A.sim()
172.
173.
174.
175. if __name__=="__main__":
176.     main()

```

modulo_shuffled.py:

```

1.  #-*- coding: utf-8 -*-
2.  """
3.  Created on Mon Jun  6 18:06:30 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  import numpy as np
9.  import pandas as pd
10. from skimage import io,color
11. from modulo_SVD import SVD_raw,rescale
12. from modulo_PSNR import PSNR
13.
14. class shuffler:
15.     """
16.     Permuta los componentes de un vector, guardando la transformación
17.     para poder la inversa.
18.     """
19.     def __init__(self,n):
20.         self.dir=pd.DataFrame({'A':np.array(range(n)),
21.                                'B':np.random.permutation(range(n))})
22.
23.     def shuffle(self,A):
24.         A=np.array(A)
25.         return A[self.dir['B']]
26.
27.     def inverse(self,B):
28.         B=np.array(B)
29.         INV=self.dir.sort_values(by='B')
30.         return B[INV['A']]
31.
32. class Blocker:
33.     """
34.     Divide y reconstruye una imagen en bloques, el número de bloques por
35.     dimensión será el divisor más cercano a la raíz de dicha dimensión.
36.     """
37.     def __init__(self,X):
38.         nd=self.NearestDiv(X)
39.         self.a=nd[0]
40.         self.b=nd[1]
41.         self.shape=X.shape
42.
43.     def NearestDiv(self,X):
44.         nd=list()
45.         for d in X.shape[:2]:
46.             divs=np.array([i for i in range(1,d+1) if d % i==0])

```

```

47.         idx=np.argmin(np.abs(divs-d**0.5))
48.         nd.append(divs[idx])
49.     return nd
50.
51.     def toBlocks(self,X):
52.         m,n=X.shape[:2]
53.         self.step_m=m//self.a
54.         self.step_n=n//self.b
55.         chain=list()
56.         for i in range(0,m,self.step_m):
57.             for j in range(0,n,self.step_n):
58.                 chain.append(X[i:(i+self.step_m), j:(j+self.step_n)])
59.         return chain
60.
61.     def fromBlocks(self,chain):
62.         A=np.zeros(self.shape,dtype='uint8')
63.         idx=list()
64.         for i in range(0,self.shape[0],self.step_m):
65.             for j in range(0,self.shape[1],self.step_n):
66.                 idx.append((i,j))
67.
68.         for n,d in enumerate(idx):
69.             i,j=d
70.
71.             A[i:(i+self.step_m),j:(j+self.step_n)]=chain[n]
72.
73.         return A
74.
75. class SSVD(SVD_raw):
76.     """
77.     Método en el que se permutan los bloques de la imagen y se ejecuta la
78.     SVD
79.     """
80.     def __init__(self,img):
81.
82.         self.B=Blocker(img)
83.         blocks=self.B.toBlocks(img)
84.         self.SH=shuffler(len(blocks))
85.         blocks_S=self.SH.shuffle(blocks)
86.         img_S=self.B.fromBlocks(blocks_S)
87.
88.         SVD_raw.__init__(self,img_S)
89.
90.     def trunc(self,k):
91.
92.         truncated=SVD_raw.trunc(self,k)
93.         blocks_S=self.B.toBlocks(truncated)
94.         blocks=self.SH.inverse(blocks_S)
95.         img_r=self.B.fromBlocks(blocks)
96.
97.         return img_r
98.
99.
100. def main():
101.     #RGB
102.     img=img=io.imread('../GCD.PNG')[:, :, :3]
103.     A=PSNR(img,SSVD)
104.     A.sim()
105.
106.     #gray
107.     gray=color.rgb2gray(img)
108.     gray=rescale(gray,(0,1))
109.     A=PSNR(gray,SVD_raw)
110.     A.sim()
111.
112.
113.
114. if __name__=="__main__":
115.     main()

```

modulo_SVD_bloques.py:

```
1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Mon Jun  6 17:45:29 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  from modulo_shuffled import Blocker
9.  from modulo_SVD import SVD_raw, rescale
10. import numpy as np
11. import matplotlib.pyplot as plt
12. from modulo_PSNR import PSNR
13. from skimage import io, color
14.
15. class SVD_bloques():
16.     """
17.     Divide la imagen en bloques y aplica la SVD a cada bloque.
18.     """
19.     def __init__(self, img):
20.         self.RGB = len(img.shape) > 2
21.         self.img = img
22.         self.B = Blocker(img)
23.         self.i_b = self.B.toBlocks(img)
24.
25.     def trunc(self, k):
26.         # Si sólo hay una entrada, asumimos que es la misma para todos
27.         if type(k) == int:
28.             K = np.repeat(k, self.B.a * self.B.b)
29.         # De lo contrario, habrá que tener una por cada bloque.
30.         else:
31.             K = k
32.
33.         t_b = self.i_b.copy()
34.         matrices = list()
35.         for i in range(len(t_b)):
36.             FACT = SVD_raw(self.i_b[i])
37.             t_b[i] = FACT.trunc(int(K[i]))
38.             matrices.append((FACT.u[:, :int(K[i])],
39.                             FACT.s[:int(K[i])],
40.                             FACT.v[:int(K[i]), :]))
41.         truncated = self.B.fromBlocks(t_b)
42.         return truncated
43.
44.     def plot(self, k):
45.         if type(k) == int:
46.             plt.title('Nº de Valores Singulares= %i'%(k))
47.         else:
48.             plt.title('Nº de Valores Singulares variable')
49.         if self.RGB:
50.             plt.imshow(self.trunc(k))
51.         else:
52.             plt.imshow(self.trunc(k), cmap='gray')
53.         plt.axis('off')
54.
55.     def mem(self, k):
56.         a = self.B.a
57.         b = self.B.b
58.         if type(k) == int:
59.             K = np.repeat(k, a * b)
60.         else:
61.             K = k
62.
63.         if self.RGB:
64.             rows, cols, _ = self.img.shape
65.
66.             m = np.sum(K) * 32 * (max(rows/a, cols/b) + 3 * min(rows/a, cols/b) + 1) / 8 / 1024 ** 2 # MB
67.         else:
```

```

68.         rows,cols=self.img.shape
69.         m=np.sum(K)*(rows/a+cols/b+1)*32/8/1024 #KB
70.
71.         return m
72.
73.
74. def main():
75.     #RGB
76.     img=img=io.imread('../GCD.PNG')[ :, :, 3]
77.     A=PSNR(img,SVD_bloques)
78.     A.max_k=min(A.fun.B.step_m,
79.                 A.fun.B.step_n)
80.
81.     A.sim()
82.
83.     #gray
84.     gray=color.rgb2gray(img)
85.     gray=rescale(gray,(0,1))
86.     A=PSNR(gray,SVD_bloques)
87.     A.max_k=min(A.fun.B.step_m,
88.                 A.fun.B.step_n)
89.     A.sim()
90.
91.
92. if __name__=="__main__":
93.     main()
94.

```

modulo_aprox.py:

```

1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Mon Jun  6 19:27:35 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.  from modulo_SVD import SVD_raw,rescale
8.  from skimage import io,color
9.  from modulo_PSNR import PSNR
10. from modulo_SVD_bloques import SVD_bloques
11. import numpy as np
12.
13. class RAW_aprox(SVD_raw):
14.     """
15.     Mismo procedimiento que SVD normal, sólo que se redondean los valores
16.     de las matrices de la factorización.
17.     """
18.     def __init__(self,img):
19.         SVD_raw.__init__(self,img)
20.         self.u=self.u.round(4)
21.         self.v=self.v.round(4)
22.         self.s=self.s.round()
23.
24.     def mem(self,k):
25.
26.         if self.RGB:
27.             rows,cols,_=self.img.shape
28.
29.             m= k*(max(rows,cols)+3*min(rows,cols)+2)*16/8/1024**2 #MB
30.         else:
31.             rows,cols=self.img.shape
32.             m=k*(rows+cols+2)*16/8/1024 #KB
33.
34.         return m
35.
36. class Bloques_aprox(SVD_bloques):

```

```

37.     """
38.     Mismo procedimiento que SVD en bloques, sólo que se redondean los valores
39.     de las matrices de la factorización.
40.     """
41.     def __init__(self, img):
42.         SVD_bloques.__init__(self, img)
43.
44.     def trunc(self, k):
45.         # Si sólo hay una entrada, asumimos que es la misma para todos
46.         if type(k) == int:
47.             K=np.repeat(k, self.B.a*self.B.b)
48.         # De lo contrario, habrá que tener una por cada bloque.
49.         else:
50.             K=k
51.
52.         t_b=self.i_b.copy()
53.         t_b=self.i_b.copy()
54.         matrices=list()
55.         for i in range(len(t_b)):
56.             FACT=SVD_raw(self.i_b[i])
57.             FACT.u=FACT.u.round(4)
58.             FACT.v=FACT.v.round(4)
59.             FACT.s=FACT.s.round()
60.             t_b[i]=FACT.trunc(int(K[i]))
61.             matrices.append((FACT.u[:, :int(K[i])],
62.                             FACT.s[:int(K[i])],
63.                             FACT.v[:int(K[i]), :]))
64.         truncated=self.B.fromBlocks(t_b)
65.         return truncated
66.
67.     def mem(self, k):
68.         a=self.B.a
69.         b=self.B.b
70.
71.         if type(k) == int:
72.             K=np.repeat(k, a*b)
73.         else:
74.             K=k
75.
76.         if self.RGB:
77.             rows, cols, _=self.img.shape
78.
79.             m= sum(K)*16*(max(rows/a, cols/b)+3*min(rows/a, cols/b)+2)/8/1024**2#MB
80.         else:
81.             rows, cols=self.img.shape
82.             m=sum(K)*(rows/a+cols/b+2)*16/8/1024 #KB
83.
84.         return m
85.
86.     def main():
87.         #RGB
88.         img=io.imread('../GCD.PNG')[ :, :, :3]
89.         #A=PSNR(img, RAW_approx)
90.         A=PSNR(img, Bloques_aprox)
91.         A.max_k=min(A.fun.B.step_m,
92.                     A.fun.B.step_n)
93.         A.sim()
94.
95.         #gray
96.         gray=color.rgb2gray(img)
97.         gray=rescale(gray, (0,1))
98.         #A=PSNR(gray, RAW_approx)
99.         A=PSNR(gray, Bloques_aprox)
100.        A.max_k=min(A.fun.B.step_m,
101.                    A.fun.B.step_n)
102.        A.sim()
103.
104.    if __name__=="__main__":
105.        main()
106.

```

modulo_minimize.py:

```
1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Mon Jun  6 20:37:15 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  from modulo_PSNR import PSNR
9.  import numpy as np
10. from scipy.optimize import minimize
11. import matplotlib.pyplot as plt
12.
13. class OPT_K(PSNR):
14.     """
15.     Clase pensada para optimizar k de forma no lineal, para cualquier imagen
16.     y variante SVD.
17.     """
18.     def __init__(self, img, fun, difk=False):
19.         """
20.
21.         Parameters
22.         -----
23.         img : np.array
24.             Imagen objeto.
25.         fun : class
26.             Variante de SVD.
27.         difk : bool, optional
28.             Asumir o no la misma k para las variantes
29.             por bloques. The default is False.
30.
31.         Returns
32.         -----
33.         None.
34.
35.         """
36.         PSNR.__init__(self, img, fun)
37.         if hasattr(self.fun, 'B') and difk:
38.             n=self.fun.B.a*self.fun.B.b
39.         else:
40.             n=2
41.
42.         self.k0=np.repeat(self.max_k//4-2,n)
43.         bound=(1,self.max_k)
44.         self.bounds=tuple([bound for i in range(n)])
45.
46.
47.
48.
49.
50.     def RELU_mem(self, k):
51.
52.         if self.fun.mem(k)>self.mem_lim:
53.             r=45e5*self.fun.mem(k)
54.         else:
55.             r=self.fun.mem(k)/self.mem_lim
56.         return r
57.
58.     def RELU_e(self, k):
59.         e=self.PSNR(self.fun.trunc(k))
60.         ma=50
61.         if e<36:
62.             r=5e4 - e
63.         elif e<=ma:
64.             r=ma/(ma-36)-e/(ma-36)
65.         else:
66.             r=0
67.         return r
```

```

68.
69.     def f_obj(self,k):
70.         if len(k)==2:
71.             K=int(k[0])
72.         else:
73.             K=np.array([int(i) for i in k])
74.
75.         return (self.RELU_mem(K)**2 - self.RELU_e(K)**2)**0.5
76.
77.     def callback(self,X):
78.         print([i for i in X])
79.         if len(X)==2:
80.
81.             self.fun.plot(X[0])
82.         else:
83.             self.fun.plot(X)
84.         plt.plot()
85.
86.     def exe(self,m='Powell',it=5):
87.
88.         sol=minimize(self.f_obj,
89.                      self.k0,
90.                      method=m,
91.                      bounds=self.bounds,
92.                      options={'maxiter':it,'disp':True})
93.         K=[int(k) for k in sol.x]
94.         if len(K)==2:
95.             self.fun.plot(int(K[0]))
96.             plt.show()
97.             print('Tasa de compresión: {0:.2%}'.format((1-self.RELU_mem(K[0]))))
98.             print('PSNR: %0.2f dB'%self.PSNR(self.fun.trunc(int(K[0]))))
99.         else:
100.             self.fun.plot(K)
101.             plt.show()
102.             print('Tasa de compresión: {0:.2%}'.format(1-self.RELU_mem(K)))
103.             print('PSNR: %0.2f dB'%self.PSNR(self.fun.trunc(K)))
104.         return
105.

```

WDR.py:

```

1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Thu May 12 20:36:04 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  from skimage import io,color,exposure
9.  import numpy as np
10. import matplotlib.pyplot as plt
11. import pywt
12. import re
13. from SVD import trunc
14.
15. img=io.imread('./GCD.PNG')[ :, :, 3]
16. g=color.rgb2gray(img)
17. g=exposure.rescale_intensity(g,in_range=(0,1)
18.                               ,out_range='uint8').round().astype('uint8')
19.
20. def DWT(g,n=4,w='db1'):
21.     coeffs=pywt.wavedec2(g,w,level=n)
22.     #coeff_arr,coeff_slices=pywt.coeffs_to_array(coeffs)
23.     coeff_arr,coeff_slices,coeff_shape=pywt.ravel_coeffs(coeffs)
24.
25.     return coeff_arr,coeff_slices,coeff_shape

```

```

26.
27. def inverseDWT(coef_arr,coeff_slices,coeff_shapes,w='db1'):
28.     #coeffs_filt=pywt.array_to_coeffs(coef_arr, coeff_slices,output_format='wavedec2')
29.     coeffs_filt=pywt.unravel_coeffs(coef_arr, coeff_slices, coeff_shapes,
30.                                     output_format='wavedec2')
31.     Arecon=pywt.waverec2(coeffs_filt,wavelet=w)
32.
33.     return Arecon
34.
35.
36. def bit_plane(arr,idx):
37.     '''
38.     Indices codificados empezando en 1 OJO
39.     '''
40.
41.     dif=idx.copy()+1
42.     dif[1:]=np.diff(idx+1)
43.
44.     binary=[format(i,'b') for i in dif]
45.
46.     msb=[b[0] for b in binary]
47.
48.     if all([i=='1' for i in msb]):
49.         binary=[b[1:] for b in binary]
50.
51.     results=''
52.     for i in range(len(arr)):
53.         if arr[i] >=0:
54.             results+='+'
55.         else:
56.             results+='-'
57.         results+=binary[i]
58.     return results
59.
60.
61. def new_bit(i,T):
62.     li=T
63.     hi=2*T
64.     f=(hi-li)/2+li
65.     if np.abs(i)>=f:
66.         new='1'
67.     else:
68.         new='0'
69.     return new
70.
71. def encode_WDR(coefs,loops=12,conv=2):
72.
73.
74.     #Initial stage
75.     T=np.max(coefs)*conv
76.     iniT=np.max(coefs)*conv
77.
78.     SCS='' # significant
79.     TPS=np.array([]) # temporaly significant
80.     ICS=coefs.copy() # not significant
81.
82.
83.     #loop
84.     for times in range(loops):
85.
86.         T/=conv
87.
88.         #Significant pass stage
89.
90.         idx=np.where(np.abs(ICS)>=T)
91.         values=ICS[idx]
92.         TPS=np.append(TPS,values)
93.         ICS=np.delete(ICS,idx)
94.
95.         cod=bit_plane(values, idx[0])

```

```

96.
97.     #refining stage
98.     splitted=re.split('\W\d*',cod)
99.     splitted=[s for s in splitted if s!='']
100.
101.         new=''
102.
103.         for i,c in enumerate(TPS):
104.             subcod=splitted[i]
105.             new+=subcod[0]+new_bit(c,T)+subcod[1:]
106.
107.
108.         SCS+=new + '|'
109.
110.         TPS=np.array([])
111.
112.
113.
114.     return (int(iniT),SCS)
115.
116.
117. def PAD(a,b):
118.     r=a.copy()
119.     z=np.where(a==0)[0]
120.     if len(z)>len(b):
121.         r[z:len(b)]=b
122.     else:
123.         r[z]=b[:len(z)]
124.         r=np.append(r,b[len(z):])
125.     return r
126.
127.
128. def decode_WDR(iniT,bitstream, slices,n=12,conv=2):
129.     k=sorted([iniT/(conv*i) for i in range(1,n+1)])
130.
131.     bitsplit=re.split('|',bitstream)[:n]
132.     bitsplit=bitsplit[:-1] #Le damos la vuelta ya que tiene que ir acorde con K
133.
134.     array=0
135.     for i in range(n):
136.         subcod=re.split('\W\d*',bitsplit[i])
137.         subcod=[s for s in subcod if s!='']
138.         diff=[]
139.         value=[]
140.         for j in subcod:
141.             if j[0]=='+' :
142.                 s=1
143.             elif j[0]=='-' :
144.                 s=-1
145.             if j[1]=='0':
146.                 value.append(s*k[i])
147.             elif j[1]=='1':
148.                 value.append(s*k[i]*1.5)
149.
150.             diff.append(int('1'+j[2:],2))
151.         idx=np.cumsum(diff)-1
152.         new_array=np.zeros(np.max(idx)+1)
153.         new_array[idx]=value
154.         if i!=0:
155.             array=PAD(new_array,array)
156.         else:
157.             array=new_array.copy()
158.     #Zero padding
159.     orig_len=slices[-1]['dd'].stop
160.     array=np.append(array,np.zeros(orig_len-len(array)))
161.     return array
162.
163.
164.
165. def ALL(g,loops=12,conv=2,w='db1',nw=2,svd=135):

```

```

166.     A=trunc(svd,g)
167.     plt.imshow(A,cmap='gray')
168.     plt.show()
169.     coefs,slices,shapes=DWT(A,nw,w)
170.     iniT, bitstream =encode_WDR(coefs,loops,conv)
171.
172.     arr=decode_WDR(iniT, bitstream,slices,loops,conv)
173.
174.     Z=inverseDWT(arr,slices,shapes)
175.
176.     fig,ax=plt.subplots(2,2,sharey='row',sharex='row')
177.     ax[0,0].plot(coefs)
178.     ax[0,1].plot(arr)
179.     ax[1,0].imshow(g,cmap='gray')
180.     ax[1,1].imshow(Z,cmap='gray')
181.     ax[1,0].set_axis_off()
182.     ax[1,1].set_axis_off()
183.     plt.show()
184.
185.     return Z
186.

```

HUFFMAN.py:

```

1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Thu May 12 20:36:04 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  from skimage import io,color,exposure
9.  import numpy as np
10. import matplotlib.pyplot as plt
11. import pywt
12. import re
13. from SVD import trunc
14.
15. img=io.imread('../GCD.PNG')[::,:3]
16. g=color.rgb2gray(img)
17. g=exposure.rescale_intensity(g,in_range=(0,1)
18.                               ,out_range='uint8').round().astype('uint8')
19.
20. def DWT(g,n=4,w='db1'):
21.     coefs=pywt.wavedec2(g,w,level=n)
22.     #coeff_arr,coeff_slices=pywt.coeffs_to_array(coefs)
23.     coeff_arr,coeff_slices,coeff_shape=pywt.ravel_coeffs(coefs)
24.
25.     return coeff_arr,coeff_slices,coeff_shape
26.
27. def inverseDWT(coef_arr,coeff_slices,coeff_shapes,w='db1'):
28.     #coefs_filt=pywt.array_to_coeffs(coef_arr, coeff_slices,output_format='wavedec2')
29.     coefs_filt=pywt.unravel_coeffs(coef_arr, coeff_slices, coeff_shapes,
30.                                     output_format='wavedec2')
31.     Arecon=pywt.waverec2(coefs_filt,wavelet=w)
32.
33.     return Arecon
34.
35.
36. def bit_plane(arr,idx):
37.     '''
38.     Indices codificados empezando en 1 OJO
39.     '''
40.
41.     dif=idx.copy()+1
42.     dif[1:]=np.diff(idx+1)

```

```

43.
44.     binary=[format(i,'b') for i in dif]
45.
46.     msb=[b[0] for b in binary]
47.
48.     if all([i=='1' for i in msb]):
49.         binary=[b[1:] for b in binary]
50.
51.     results=''
52.     for i in range(len(arr)):
53.         if arr[i] >=0:
54.             results+='+'
55.         else:
56.             results+='-'
57.             results+=binary[i]
58.     return results
59.
60.
61. def new_bit(i,T):
62.     li=T
63.     hi=2*T
64.     f=(hi-li)/2+li
65.     if np.abs(i)>=f:
66.         new='1'
67.     else:
68.         new='0'
69.     return new
70.
71. def encode_WDR(coefs,loops=12,conv=2):
72.
73.
74.     #Initial stage
75.     T=np.max(coefs)*conv
76.     iniT=np.max(coefs)*conv
77.
78.     SCS='' # significant
79.     TPS=np.array([]) # temporaly significant
80.     ICS=coefs.copy() # not significant
81.
82.
83.     #loop
84.     for times in range(loops):
85.
86.         T/=conv
87.
88.         #Significant pass stage
89.
90.         idx=np.where(np.abs(ICS)>=T)
91.         values=ICS[idx]
92.         TPS=np.append(TPS,values)
93.         ICS=np.delete(ICS,idx)
94.
95.         cod=bit_plane(values, idx[0])
96.
97.         #refining stage
98.         splitted=re.split('(\W\d*)',cod)
99.         splitted=[s for s in splitted if s!='']
100.
101.         new=''
102.
103.         for i,c in enumerate(TPS):
104.             subcod=splitted[i]
105.             new+=subcod[0]+new_bit(c,T)+subcod[1:]
106.
107.
108.         SCS+=new + '|'
109.
110.         TPS=np.array([])
111.
112.

```

```

113.
114.     return (int(iniT),SCS)
115.
116.
117. def PAD(a,b):
118.     r=a.copy()
119.     z=np.where(a==0)[0]
120.     if len(z)>=len(b):
121.         r[z[:len(b)]] = b
122.     else:
123.         r[z]=b[:len(z)]
124.         r=np.append(r,b[len(z):])
125.     return r
126.
127.
128. def decode_WDR(iniT,bitstream, slices,n=12,conv=2):
129.     k=sorted([iniT/(conv**i) for i in range(1,n+1)])
130.
131.     bitsplit=re.split('\|',bitstream)[: -1]
132.     bitsplit=bitsplit[: -1] #Le damos la vuelta ya que tiene que ir acorde con K
133.
134.     array=0
135.     for i in range(n):
136.         subcod=re.split('(\W\d*)',bitsplit[i])
137.         subcod=[s for s in subcod if s!='']
138.         diff=[]
139.         value=[]
140.         for j in subcod:
141.             if j[0]=='+' :
142.                 s=1
143.             elif j[0]=='-' :
144.                 s=-1
145.             if j[1]=='0':
146.                 value.append(s*k[i])
147.             elif j[1]=='1':
148.                 value.append(s*k[i]*1.5)
149.
150.             diff.append(int('1'+j[2:],2))
151.         idx=np.cumsum(diff)-1
152.         new_array=np.zeros(np.max(idx)+1)
153.         new_array[idx]=value
154.         if i!=0:
155.             array=PAD(new_array,array)
156.         else:
157.             array=new_array.copy()
158.     #Zero padding
159.     orig_len=slices[-1]['dd'].stop
160.     array=np.append(array,np.zeros(orig_len-len(array)))
161.     return array
162.
163.
164.
165. def ALL(g,loops=12,conv=2,w='db1',nw=2,svd=135):
166.     A=trunc(svd,g)
167.     plt.imshow(A,cmap='gray')
168.     plt.show()
169.     coefs,slices,shapes=DWT(A,nw,w)
170.     iniT, bitstream =encode_WDR(coefs,loops,conv)
171.
172.     arr=decode_WDR(iniT, bitstream,slices,loops,conv)
173.
174.     Z=inverseDWT(arr,slices,shapes)
175.
176.     fig,ax=plt.subplots(2,2,sharey='row',sharex='row')
177.     ax[0,0].plot(coefs)
178.     ax[0,1].plot(arr)
179.     ax[1,0].imshow(g,cmap='gray')
180.     ax[1,1].imshow(Z,cmap='gray')
181.     ax[1,0].set_axis_off()
182.     ax[1,1].set_axis_off()

```

```

183.     plt.show()
184.
185.     return Z
186.

```

HUFFMAN 2.py:

```

1.  # -*- coding: utf-8 -*-
2.  """
3.  Created on Sat May 14 13:50:42 2022
4.
5.  @author: David Garcerán Herráiz
6.  """
7.
8.  from WDR import encode_WDR, decode_WDR, DWT, inverseDWT
9.  from HUFFMAN import HUFF
10. from SVD import trunc
11. from PSNR_gray import PSNR
12. from skimage import io, color, exposure
13. import bitstring
14. import matplotlib.pyplot as plt
15.
16. img=io.imread('./GCD.PNG')[:, :, :3]
17. g=color.rgb2gray(img)
18. g=exposure.rescale_intensity(g, in_range=(0,1)
19.                               , out_range='uint8').round().astype('uint8')
20. g_t=trunc(150,g)
21. coefs, slices, shapes=DWT(g_t, 2, 'db1')
22. iniT, bitstream =encode_WDR(coefs,12,2)
23.
24. H=HUFF(bitstream)
25. e=H.encode(bitstream)
26.
27. f=open('prueba.bin','wb')
28. e.tofile(f)
29. f.close()
30. #12.2 KB
31.
32. d=bitstring.BitArray(filename='./prueba.bin')
33. bit=H.decode(d)
34. array=decode_WDR(iniT, bit, slices)
35. I=inverseDWT(array, slices, shapes)
36. plt.imshow(I, cmap='gray')
37. plt.title(PSNR(g, I[:, :297]))
38.

```


REFERENCIAS

- Compton, E. A., & Stacey L. Ernstberger, P. (2020). Singular Value Decomposition: Applications to Image Processing. *Lagrange College*, 17, 7.
- Jaradat, Y., Masoud, M., Jannoud, I., Manasrah, A., & Alia, M. (2021). A Tutorial on Singular Value Decomposition with Applications on Image Compression and Dimensionality Reduction. *International Conference on Information Technology (ICIT)*, 1-4.
- Javier Martínez, J. (Enero de 2005). *La descomposición en valores singulares (SVD) y algunas de sus aplicaciones*. Obtenido de ResearchGate:
https://www.researchgate.net/publication/263583897_La_descomposicion_en_valores_singulares_SVD_y_algunas_de_sus_aplicaciones
- Kuang, Z. (n.d.). *Singular-Value Decomposition and its Applications*. Undergraduate Honor Thesis, University of California, Department of Mathematics, San Diego.
- Ochoa, H., & Rao, K. (2003). A Hybrid DWT-SVD Image-Coding System (HDWTSVD) for Color Images. *Systemics, Cybernetics and Informatics*, 1, 2-64.
- Powell, M. J. (1964). An efficient method for finding the minimum of a function of several variables without calculating derivatives. *Computer Journal*, 7(2), 155-162.
- Prasanth, D. H. (Agosto de 2020). Novel Approach for Image Compression using Modified SVD. *International Journal of Creative Research Thoughts (IJCRT)*, 8(8), 2234-2243.
- PSNR. (2 de Agosto de 2019). *Wikipedia, La enciclopedia libre*. Obtenido de <https://es.wikipedia.org/w/index.php?title=PSNR&oldid=117935779>
- Ranade, A., Mahabalarao, S. S., & Kale, S. (2007). A variation on SVD based image compression. *Image and Vision computing*, 25(6), 771-777.
- Rufai, A. M., Anbarjafari, G., & Demirel, H. (2013). Lossy medical image compression using Huffman coding and singular value decomposition. En *2013 21st Signal Processing and Communications Applications Conference (SIU)* (págs. 1-4). IEEE.
doi:10.1109/SIU.2013.6531592
- Rufai, A. M., Anbarjafari, G., & Demirel, H. (2014). Lossy image compression using singular value decomposition and wavelet difference reduction. *Digital Signal Processing*, 24, 117-123.
doi:10.1016/J.DSP.2013.09.008